Convolutional Neural Networks (CNNs)

Danna Gurari
University of Colorado Boulder
Spring 2026

BY NC SA

https://dannagurari.colorado.edu/course/neural-networks-and-deep-learning-spring-2026/

Review

- Last lecture:
  - Limitation of feedforward neural networks: supporting sequential data
  - Key insight: *recurrent* neural networks (RNNs)
  - Breakthrough: gated RNNs for dynamically deciding what to remember
  - Programming tutorial: recurrent neural networks in PyTorch
- Assignments (Canvas):
  - Lab assignment 1 due Friday (tomorrow)
  - Problem set 1 due in a week
- Questions?

Today's Topics

- Issue: input shifts cause fully connected networks to fail
- Idea: convolutional neural networks (CNNs) designed for translation invariance
- Implementation: training CNNs end-to-end with gradient descent
- Real-world success: LeNet-5 for check reading
- Programming tutorial: convolutional neural networks in PyTorch

Today's Topics

- Issue: input shifts cause fully connected networks to fail
- Idea: convolutional neural networks (CNNs) designed for translation invariance
- Implementation: training CNNs end-to-end with gradient descent
- Real-world success: LeNet-5 for check reading
- Programming tutorial: convolutional neural networks in PyTorch

Context: Visual Analysis Skills of Animals

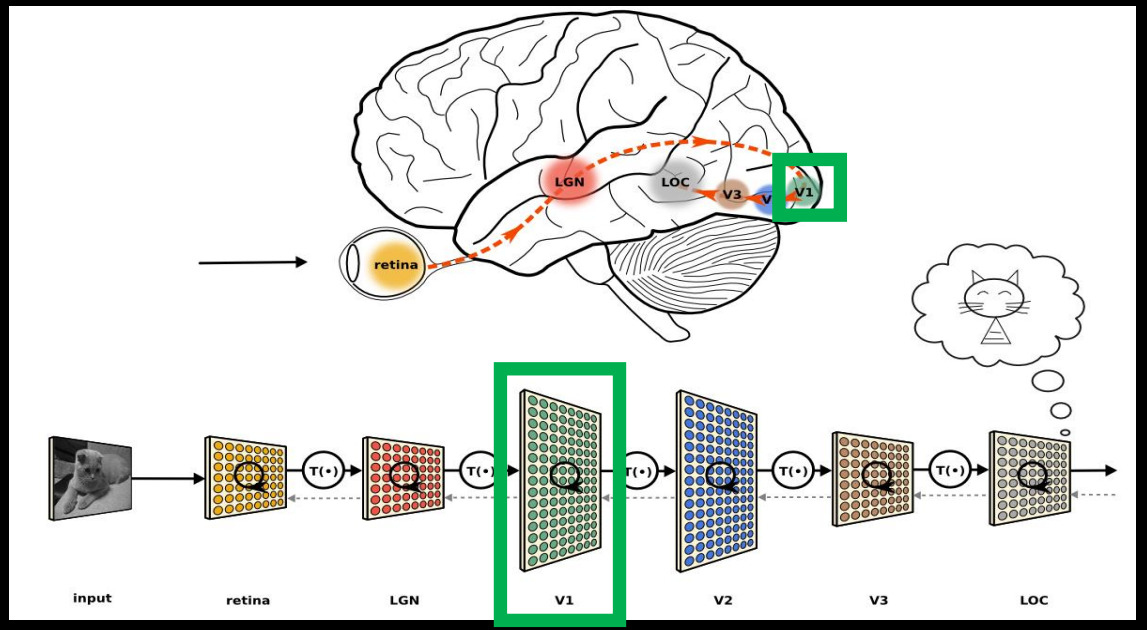


- What number is shown in the image?
  - Input is just translated
- 1981 Nobel prize winning work from 1960s explains how we do this so effortlessly!



https://braintour.harvard.edu/archives/portfolio-items/hubel-and-wiesel

David Hubel and Torsten Wiesel
(Neurophysiologists)



https://neuwritesd.files.wordpress.com/2015/10/visual_stream_small.png

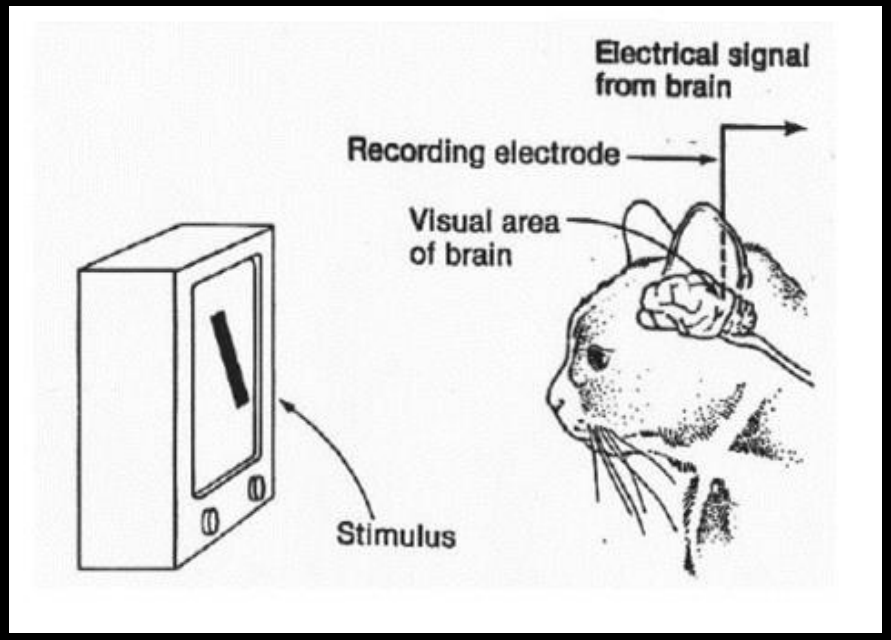
Studied V1 region of a brain's visual processing system

Context: Visual Analysis Skills of Animals



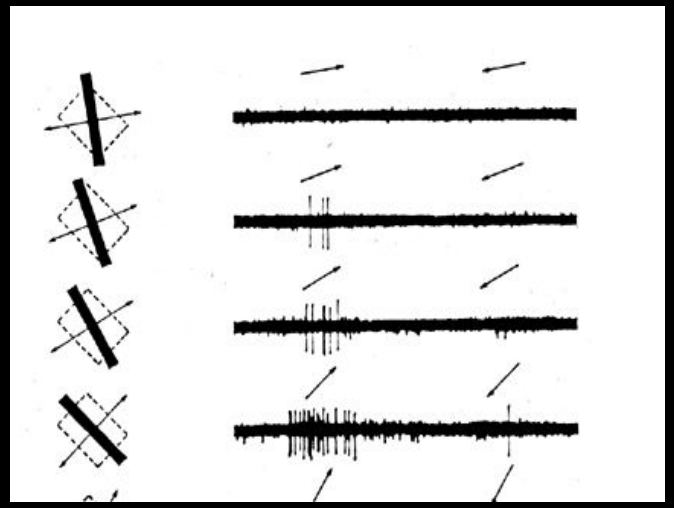
- What number is shown in the image?
  - Input is just translated
- 1981 Nobel prize winning work from 1960s explains how we do this so effortlessly!

Set-up:



https://www.esantus.com/blog/2019/1/31/convolutional-neural-networks-a-quick-guide-for-newbies

Finding 1: simple cells fire for light stimuli orientations at specific locations



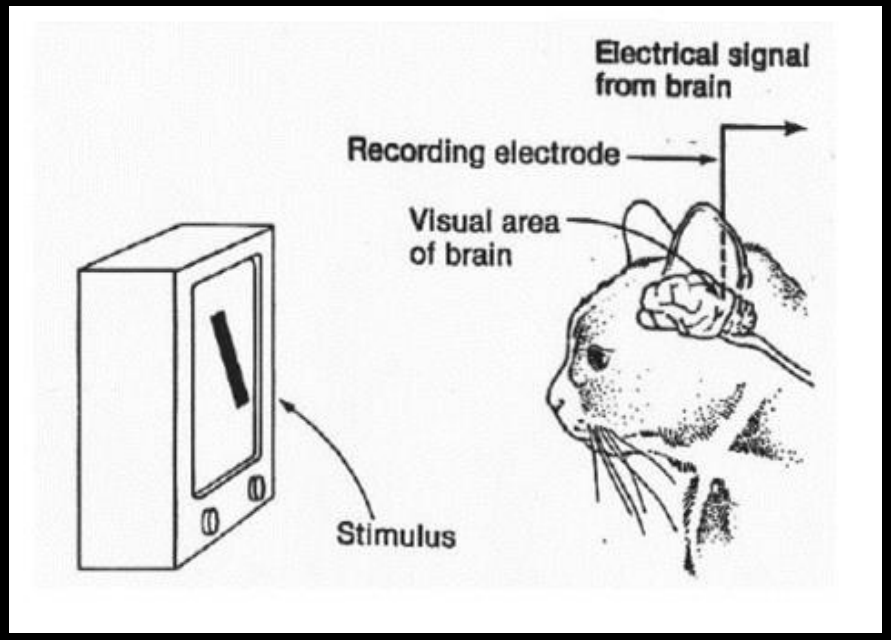
https://www.cns.nyu.edu/~david/courses/perception/lecturenotes/V1/lgn-V1.html

Context: Visual Analysis Skills of Animals



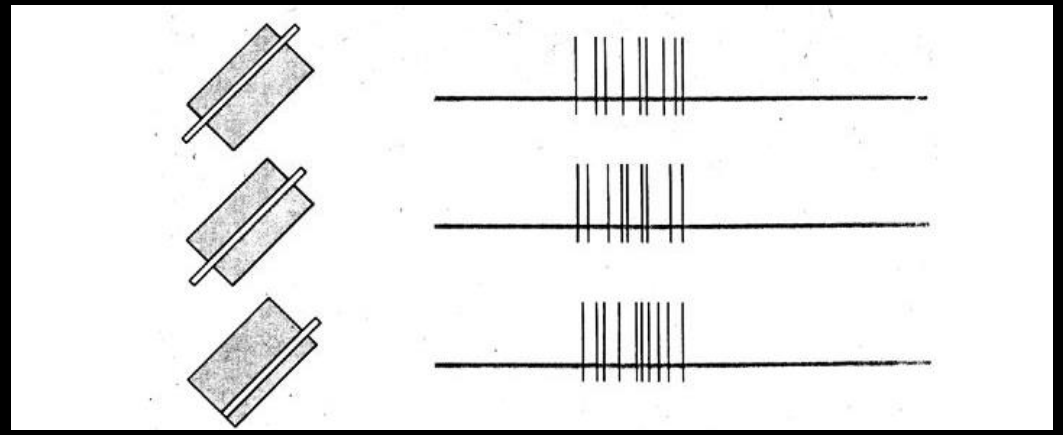
- What number is shown in the image?
  - Input is just translated
- 1981 Nobel prize winning work from 1960s explains how we do this so effortlessly!

Set-up:



https://www.esantus.com/blog/2019/1/31/convolutional-neural-networks-a-quick-guide-for-newbies

Finding 2: complex cells aggregate simple cells' responses, firing at many locations (i.e., tolerate translation)



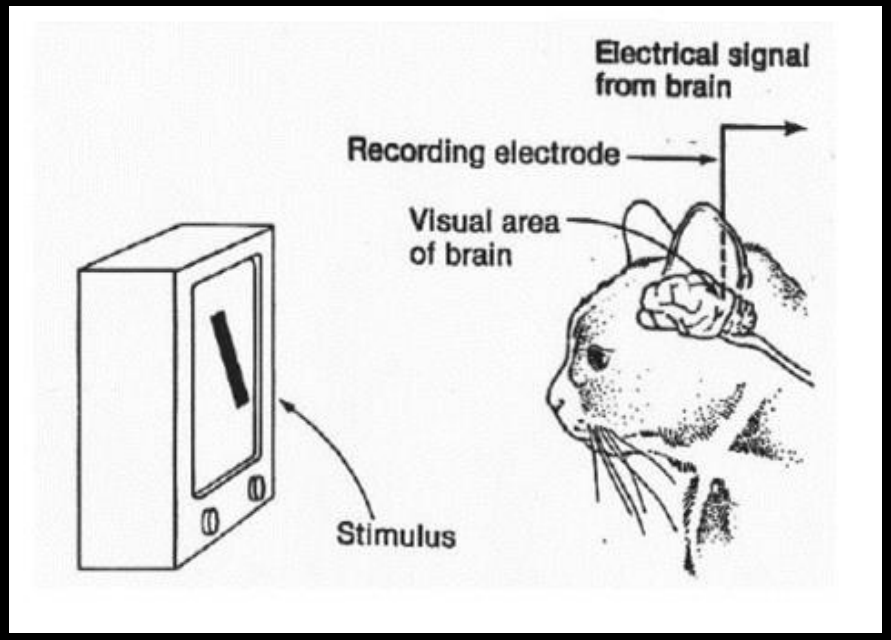
https://www.cns.nyu.edu/~david/courses/perception/lecturenotes/V1/lgn-V1.html

Context: Visual Analysis Skills of Animals

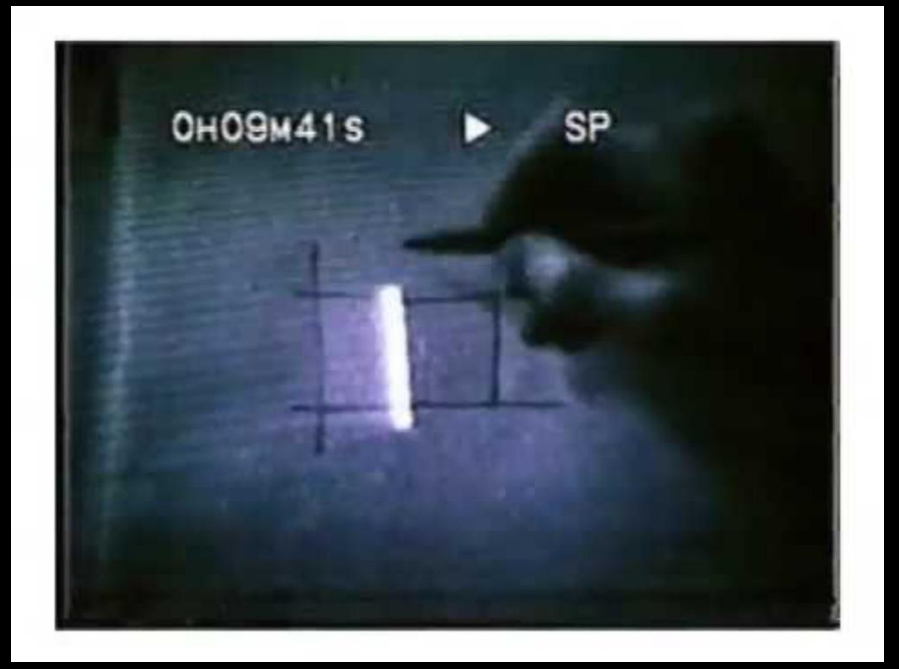


- What number is shown in the image?
  - Input is just translated
- 1981 Nobel prize winning work from 1960s explains how we do this so effortlessly!

Set-up:



https://www.esantus.com/blog/2019/1/31/convolutional-neural-networks-a-quick-guide-for-newbies

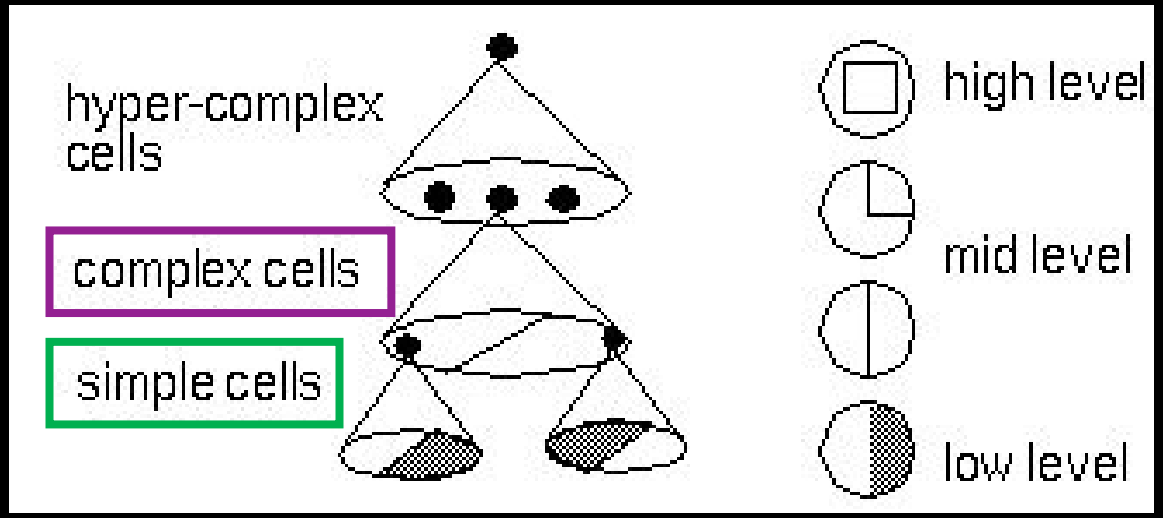


https://www.youtube.com/watch?v=OGxVfKJqX5E&ab_channel=RyanAbbott

Context: Visual Analysis Skills of Animals

- What number is shown in the image?
  - Input is just translated
- 1981 Nobel prize winning work from 1960s explains how we do this so effortlessly!





https://bruceoutdoors.files.wordpress.com/2017/08/hubel.jpg

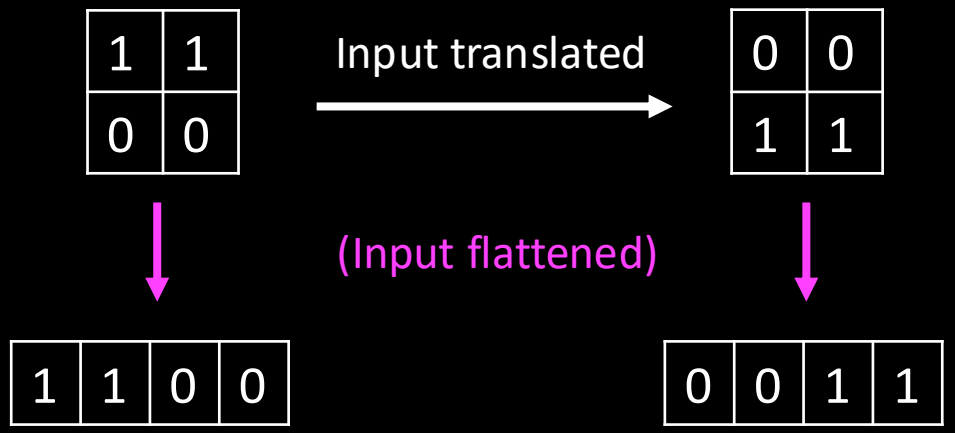
Summary: cells organized as a hierarchy of feature detectors enable translation invariance

Context: Visual Analysis Skills of Animals

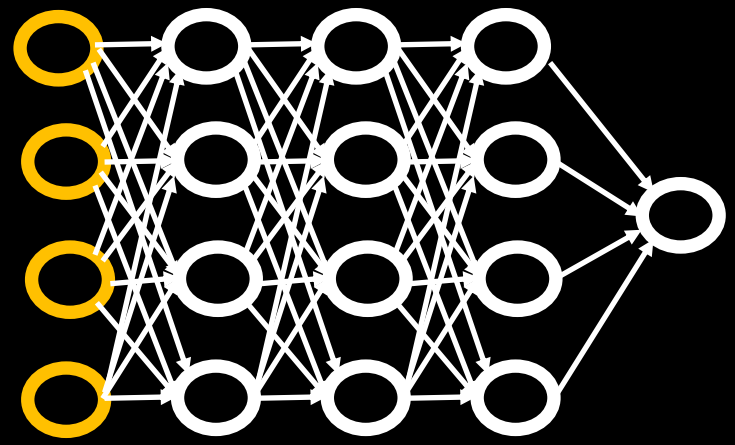
- What number is shown in the image?
  - Input is just translated



- What does a fully connected feedforward neural network see (simplified example)?



The two inputs are totally different!



Tricks/hacks needed: training data augmentation or input pre-processing

Today's Topics

- Issue: input shifts cause fully connected networks to fail
- Idea: convolutional neural networks (CNNs) designed for translation invariance
- Implementation: training CNNs end-to-end with gradient descent
- Real-world success: LeNet-5 for check reading
- Programming tutorial: convolutional neural networks in PyTorch

Goal: Translation Invariant Neural Networks

Home › Biological Cybernetics › Article

Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position

Published: April 1980

Volume 36, pages 193–202, (1980) Cite this article



http://personalpage.flsi.or.jp/fukushima/index-e.html

Fukushima Kunihiko
1936 - Present

Self-organization of a neural network which gives position-invariant response

Author: Kunihiko Fukushima | Authors Info & Claims

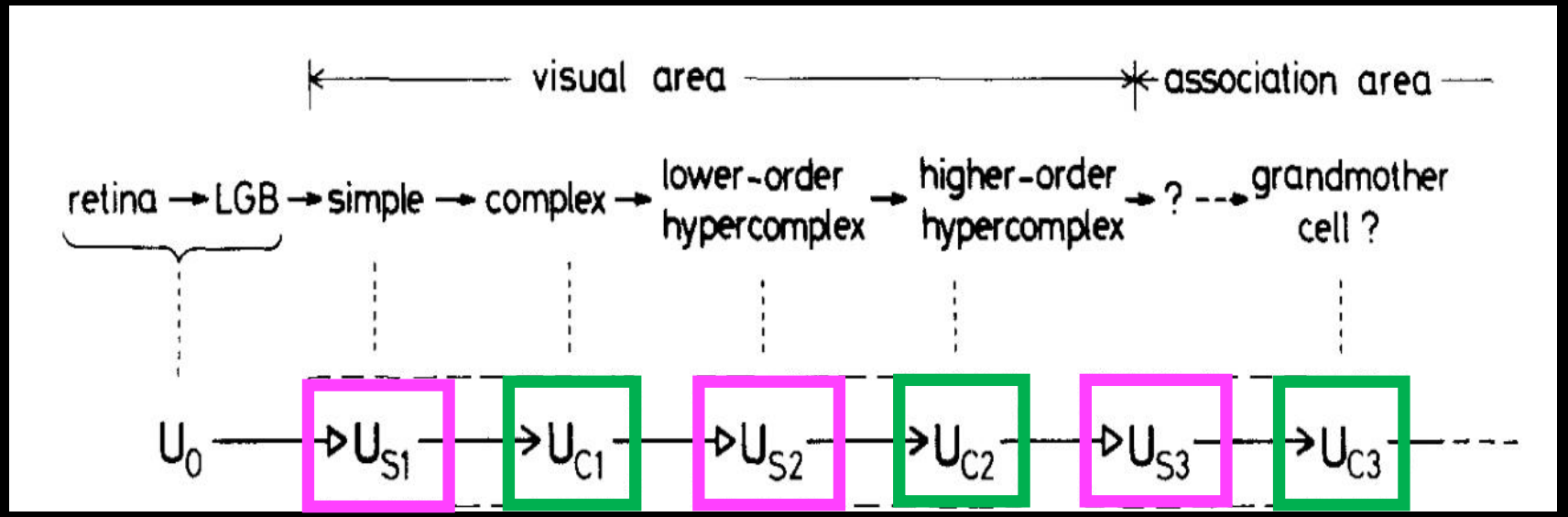
IJCAI'79: Proceedings of the 6th international joint conference on Artificial intelligence - Volume 1 • Pages 291 - 293

Published: 20 August 1979 Publication History

"…we discuss how to synthesize a neural network model in order to endow it an ability of pattern recognition like a human being… the network acquires a similar structure to the hierarchy model of the visual nervous system proposed by Hubel and Wiesel."

Approach: Convolutional Neural Networks

- Architecture is hard-coded to treat patterns as the same across spatial locations
  - This inductive bias relieves the network from needing to learn translation invariance, which is otherwise necessary for a fully connected feedforward network

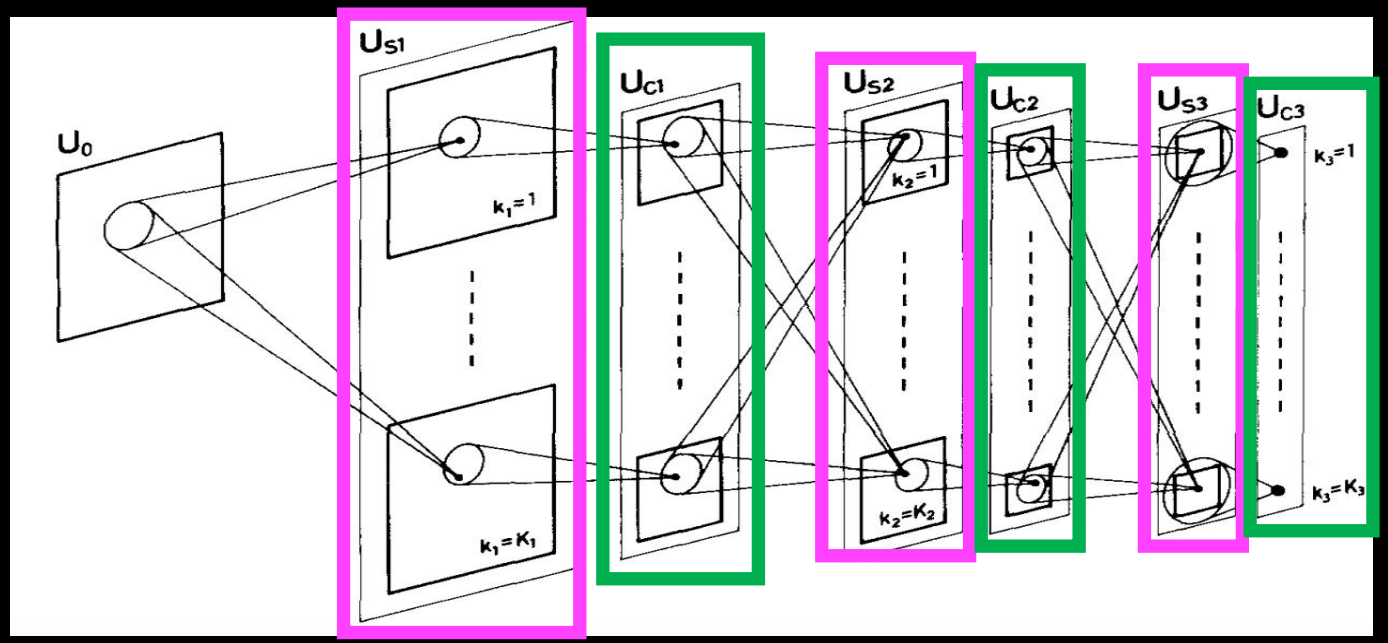


Biological design: alternating simple and complex cells

Fukushima, 1980

Approach: Convolutional Neural Networks

- Architecture is hard-coded to treat patterns as the same across spatial locations
  - This inductive bias relieves the network from needing to learn translation invariance, which is otherwise necessary for a fully connected feedforward network

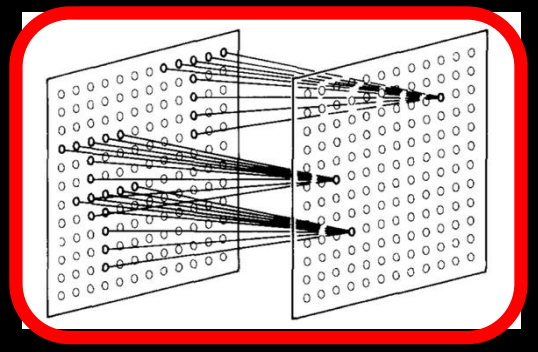


Architectural design: alternating simple and complex cells

Fukushima, 1980

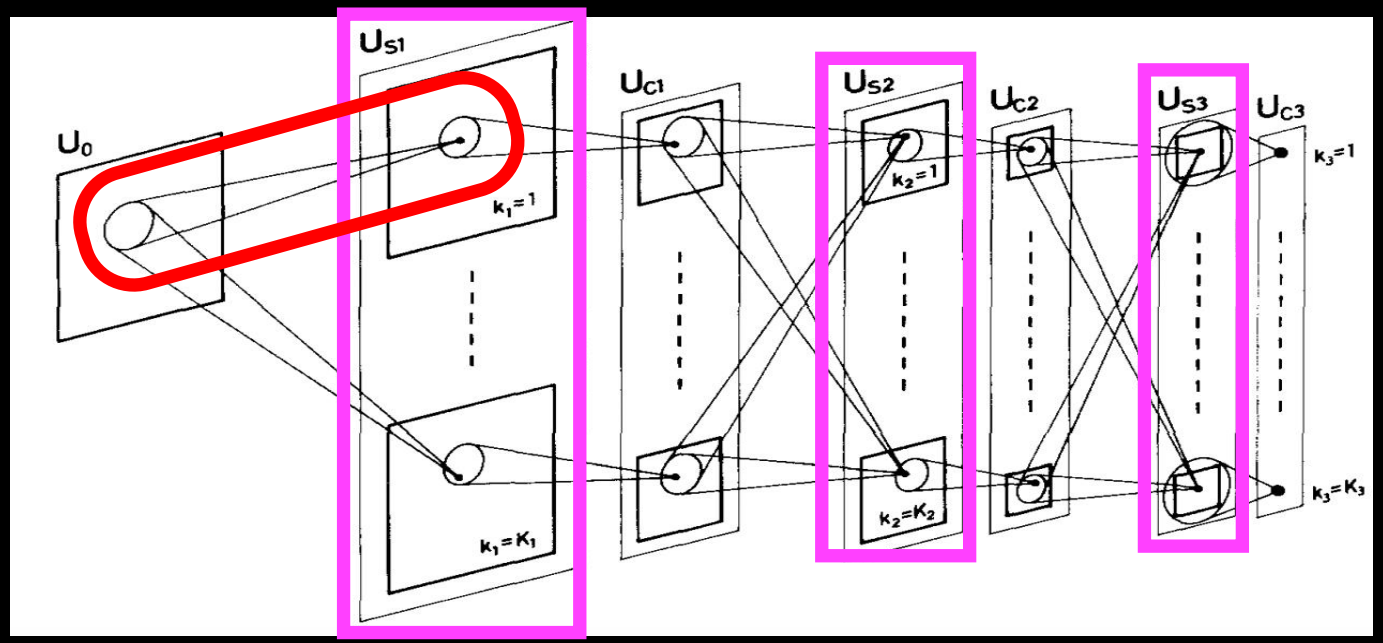
Approach: Convolutional Neural Networks

- Architecture is hard-coded to treat patterns as the same across spatial locations
  - This inductive bias relieves the network from needing to learn translation invariance, which is otherwise necessary for a fully connected feedforward network



Parallel cells at different locations fire for the same pattern; e.g., oriented line



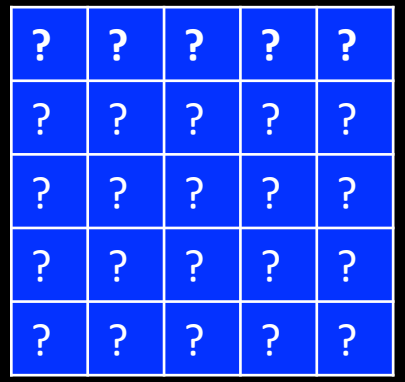


Architectural design: alternating simple and complex cells

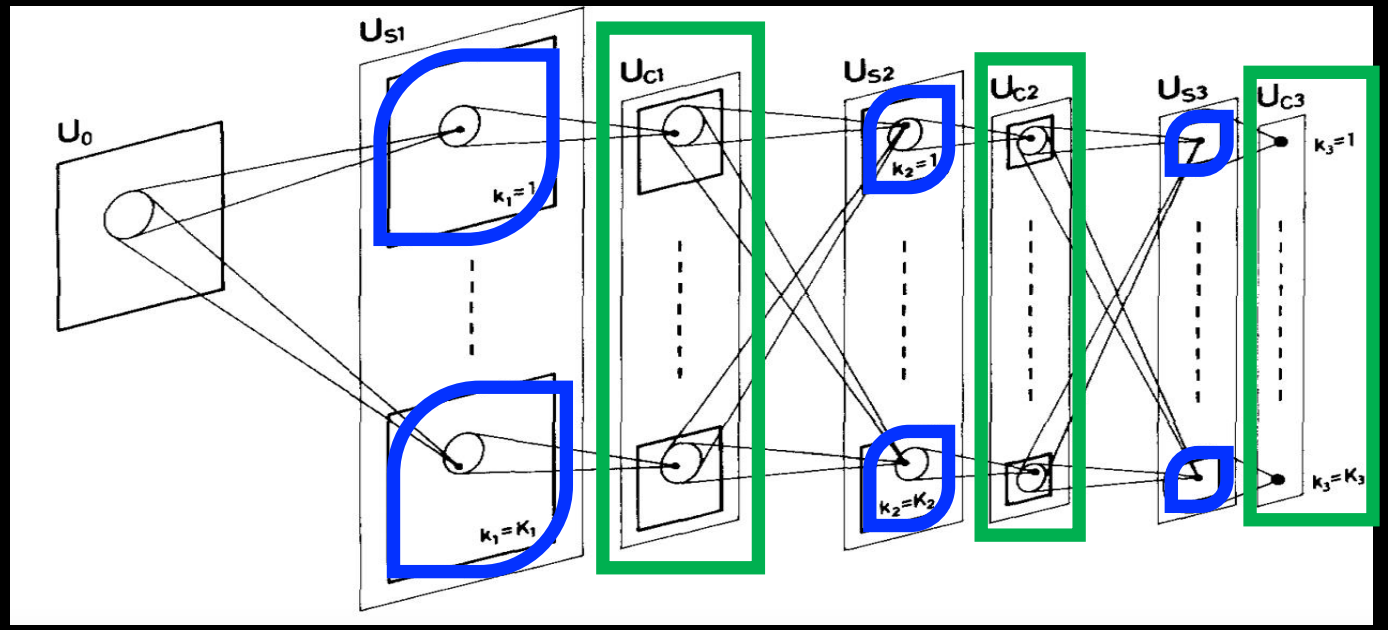
Fukushima, 1980

Approach: Convolutional Neural Networks

- Architecture is hard-coded to treat patterns as the same across spatial locations
  - This inductive bias relieves the network from needing to learn translation invariance, which is otherwise necessary for a fully connected feedforward network



Cell fires when at least one part of region has pattern (translation invariance); e.g., is any row all 1s?

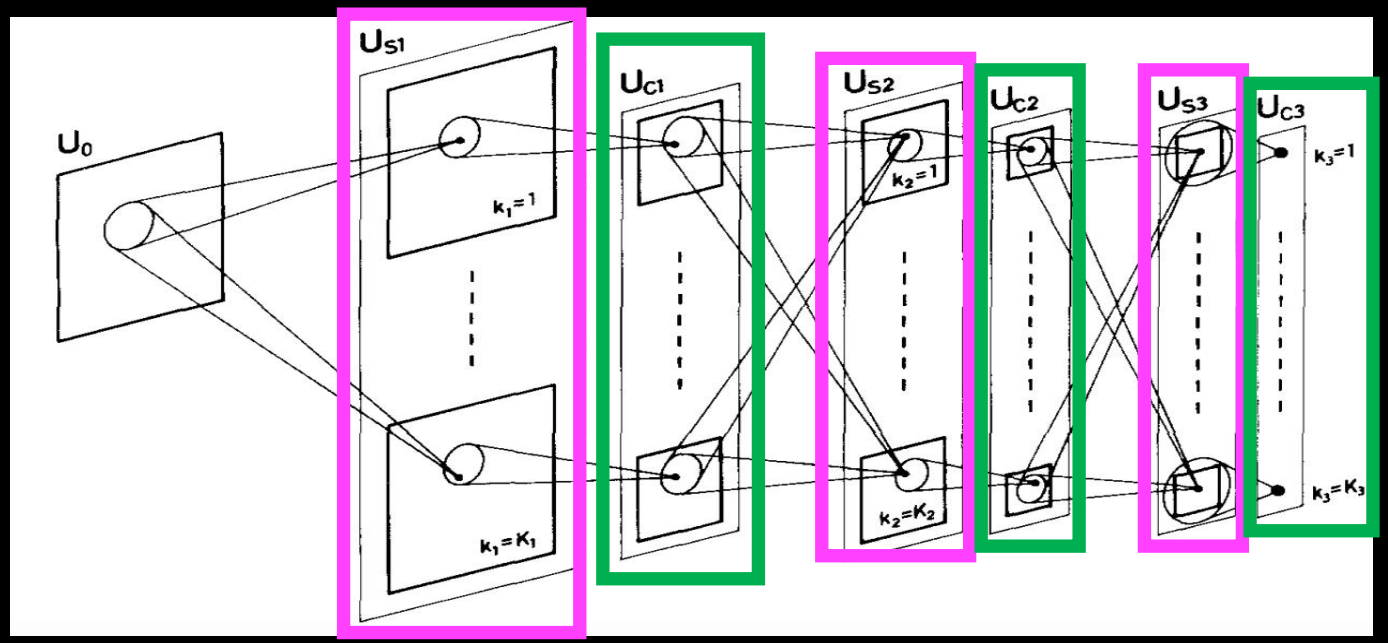


Architectural design: alternating simple and complex cells

Fukushima, 1980

Approach: Convolutional Neural Networks

- Architecture is hard-coded to treat patterns as the same across spatial locations
  - This inductive bias relieves the network from needing to learn translation invariance, which is otherwise necessary for a fully connected feedforward network



Later stages:

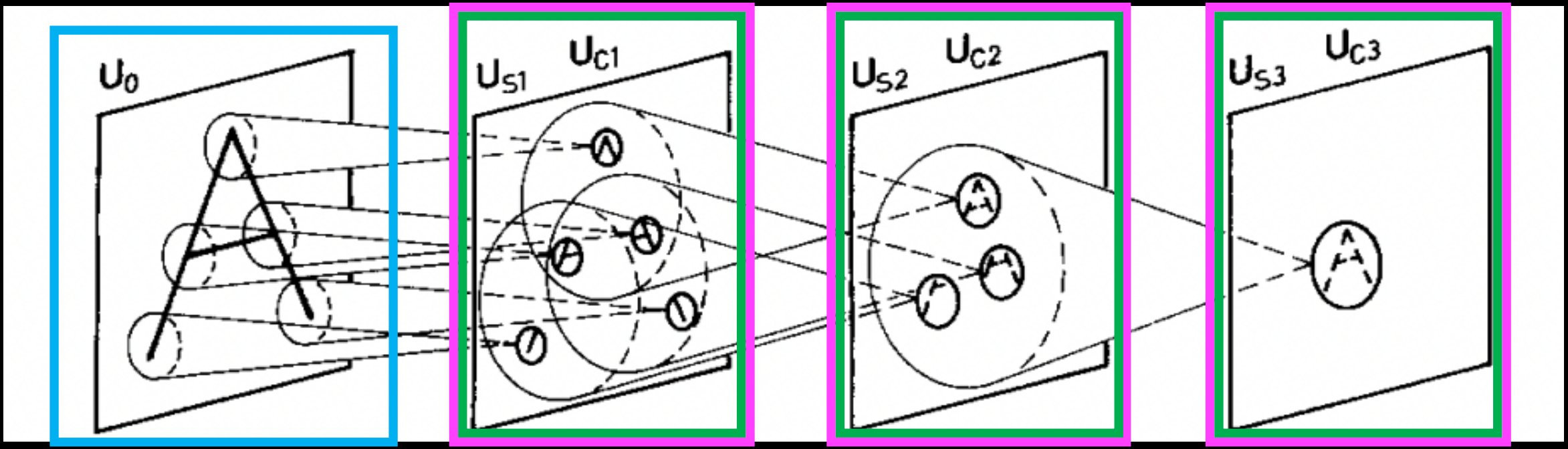
- See larger regions, so are less sensitive to pattern position shifts

- Support more complicated patterns, by combing earlier ones

Architectural design: alternating simple and complex cells

Fukushima, 1980

Illustrative Example of CNN Approach



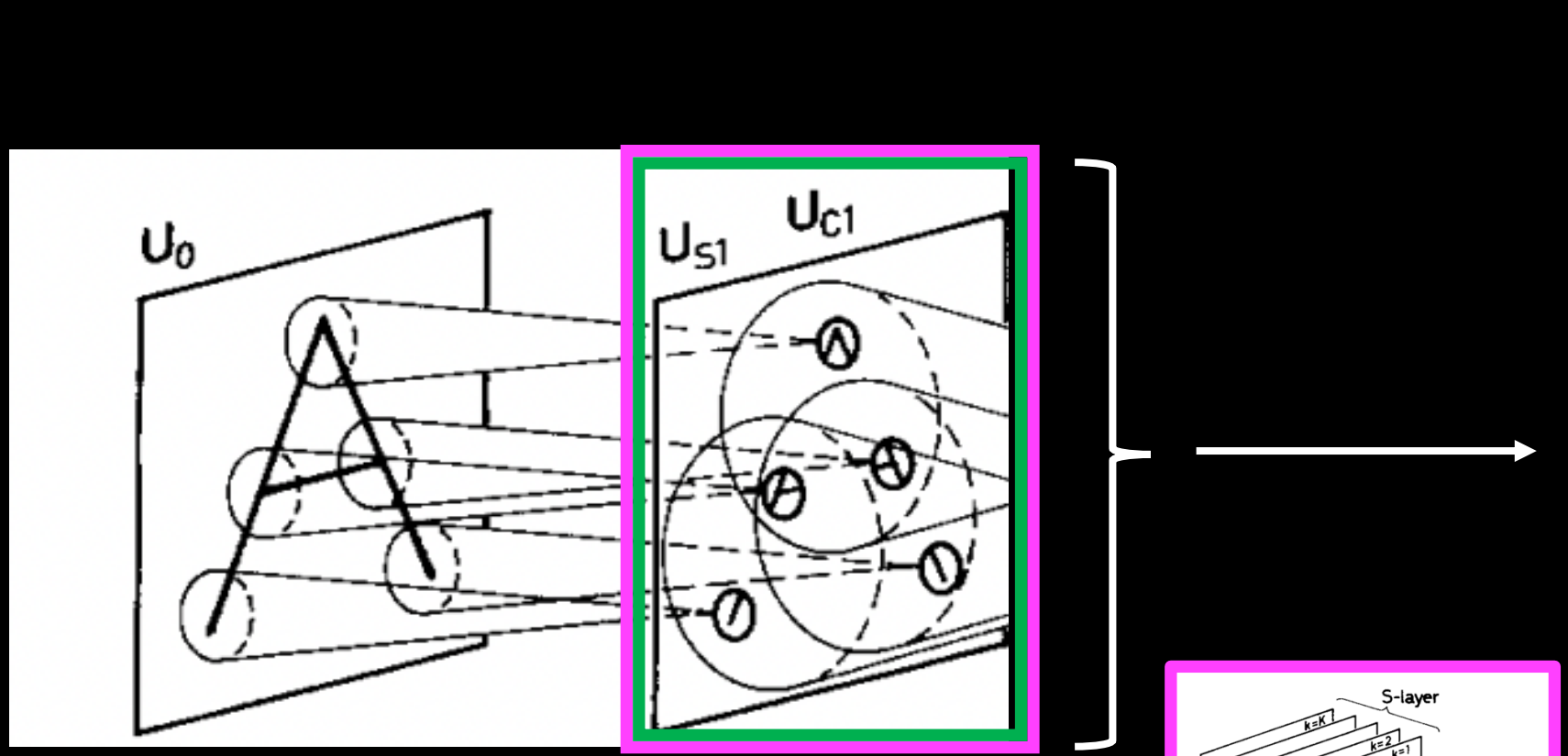
What does the input show?

Architecture: alternating simple and complex cells

What does the output show?

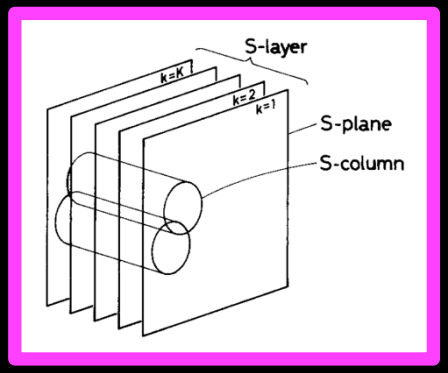
Fukushima, 1980

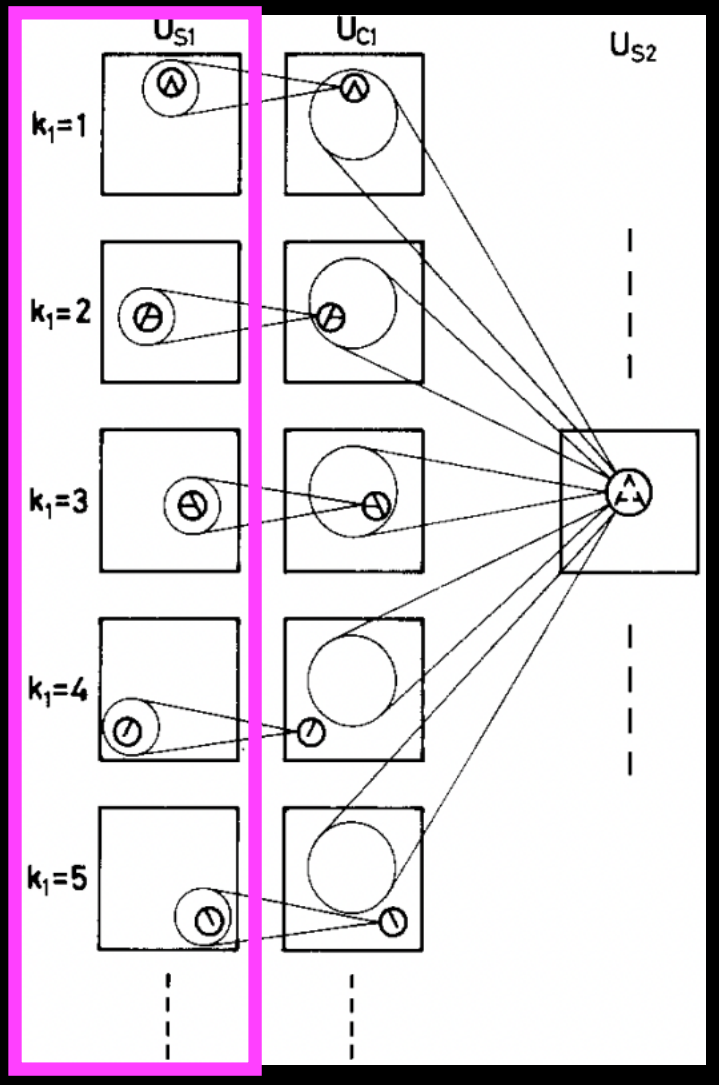
Illustrative Example of CNN Approach



Simple neurons fire for all input locations showing K patterns of interest.

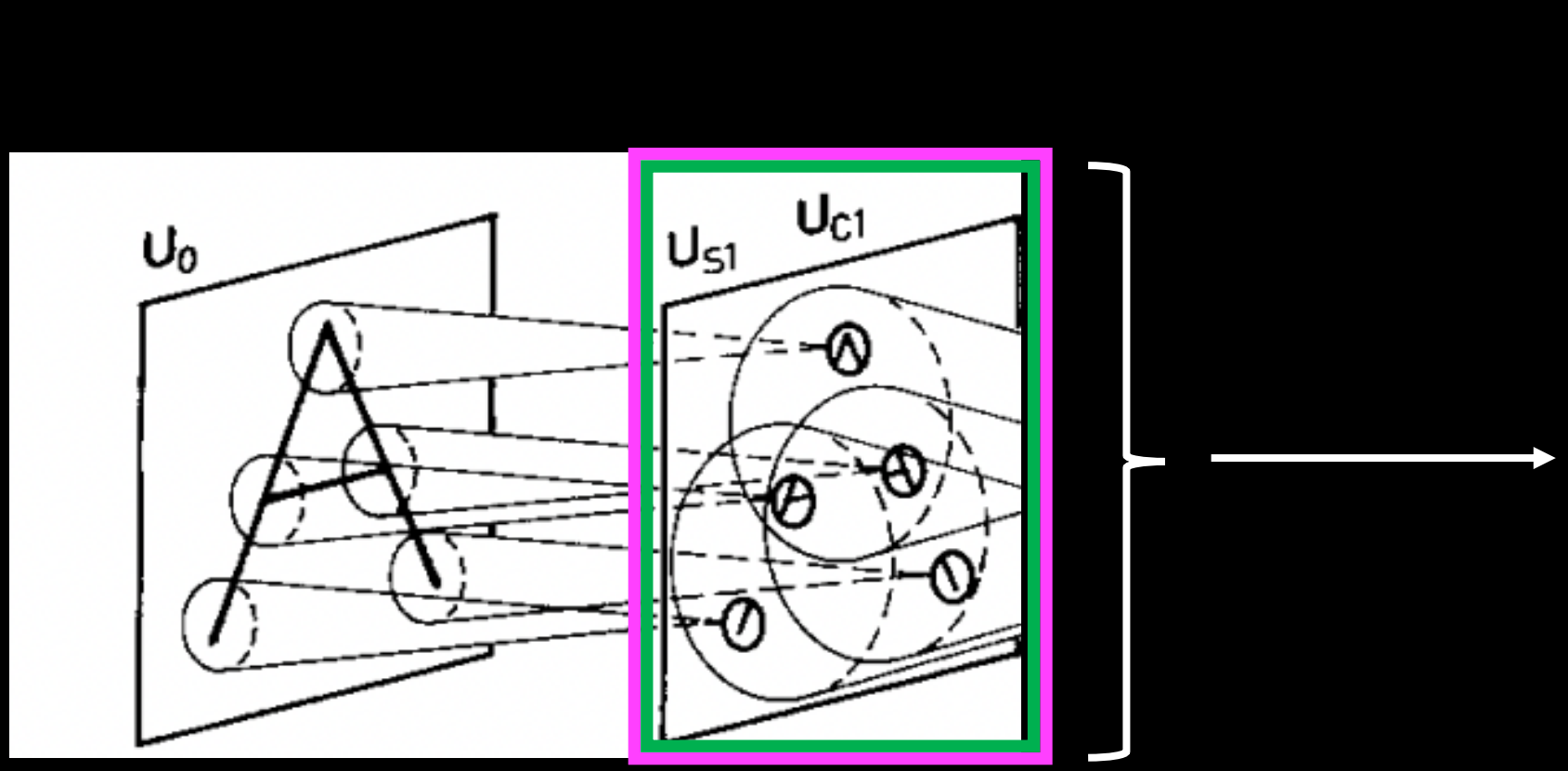
What is target pattern for k=1? k=2? k=3? …

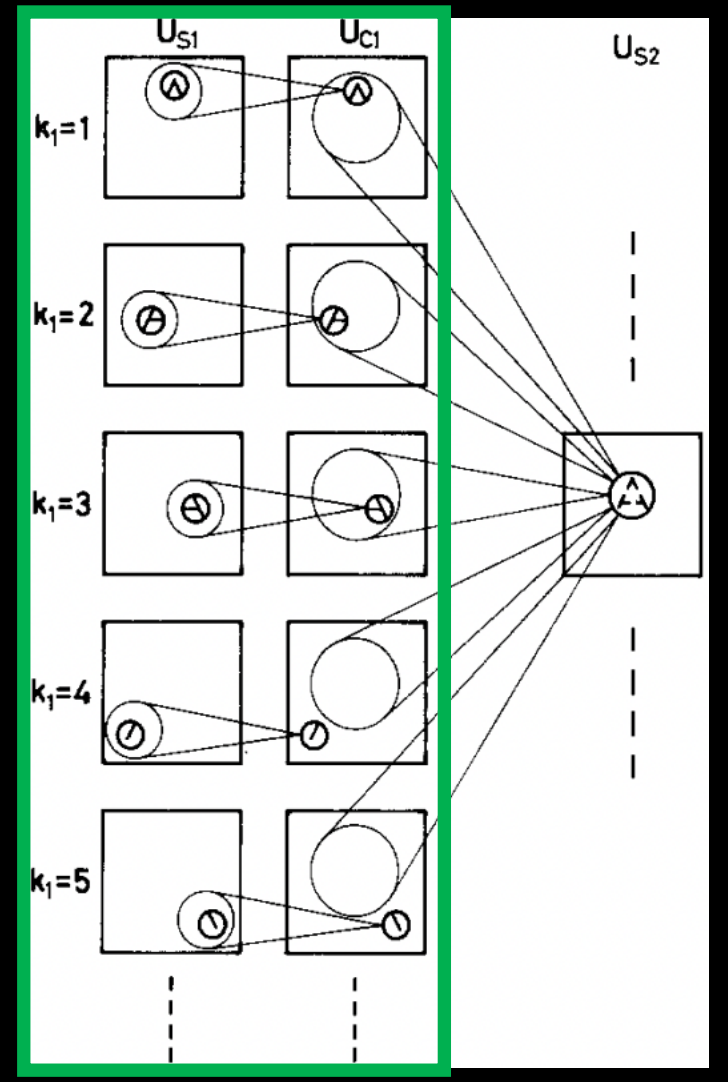




Fukushima, 1980

Illustrative Example of CNN Approach



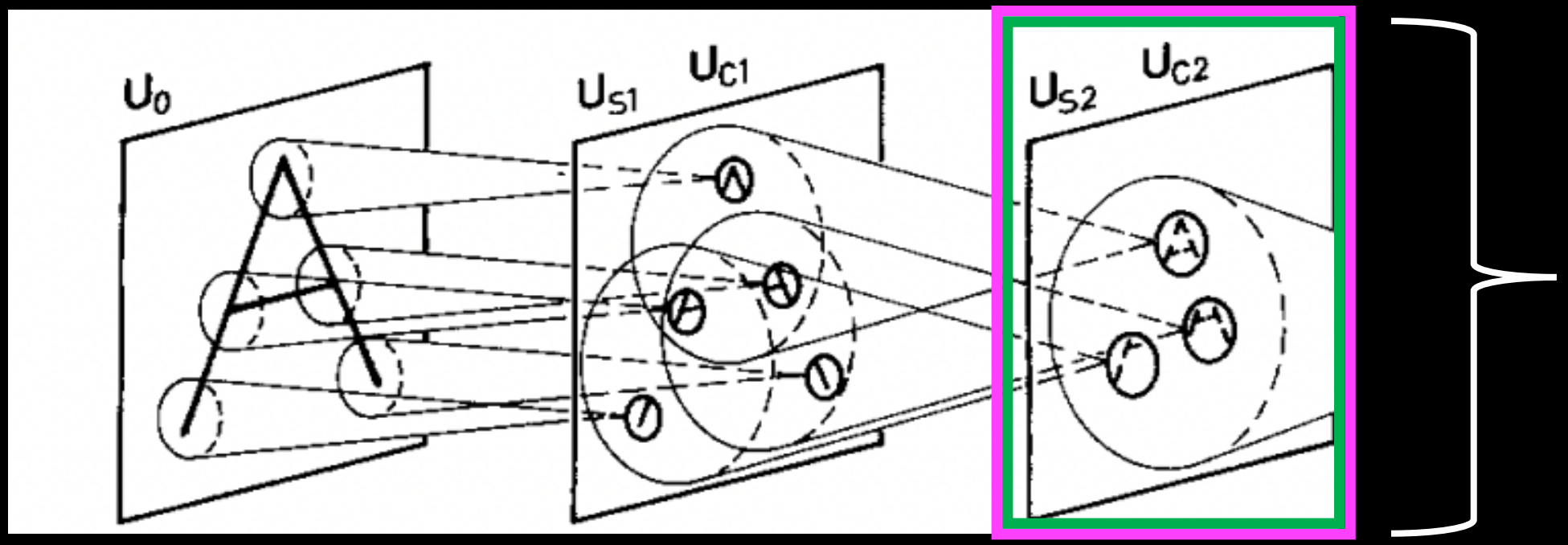


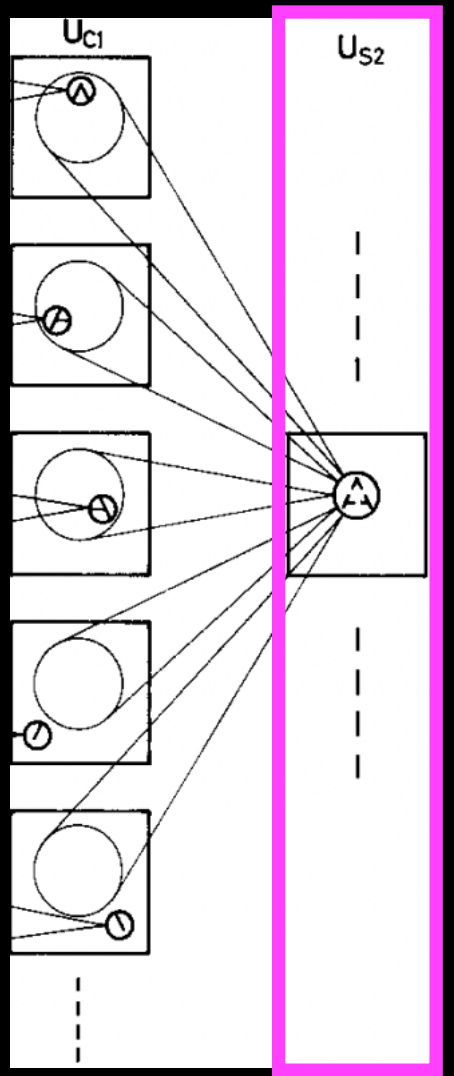
Complex neurons fire when at least one part of input target region has pattern of interest

Does it fire for k=1? k=2? k=3? …

Fukushima, 1980

Illustrative Example of CNN Approach



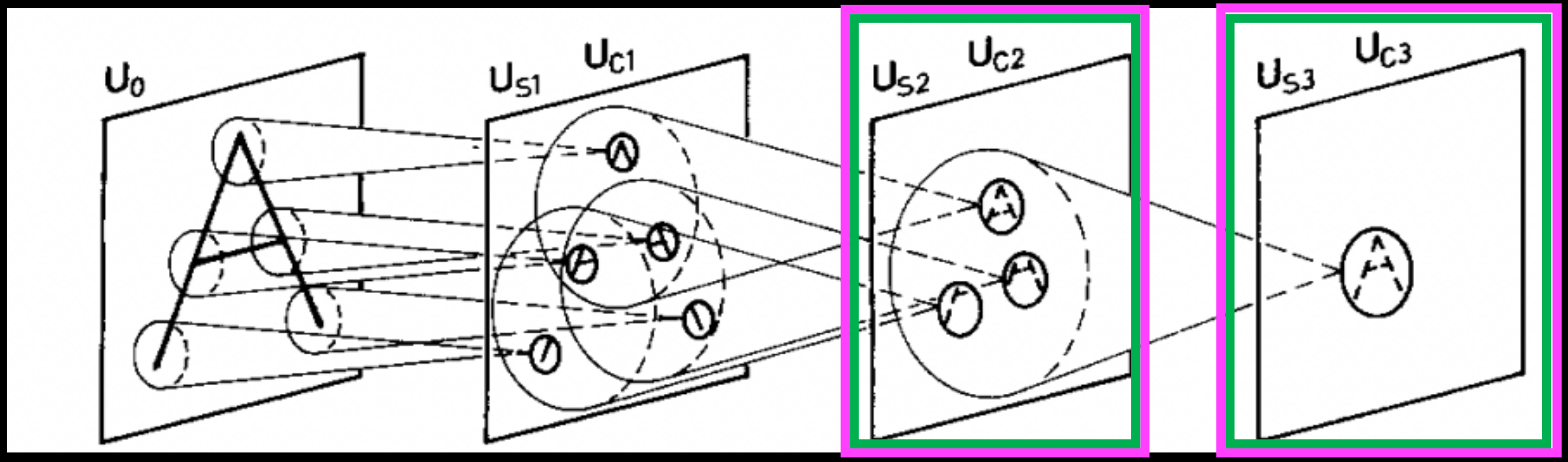


Simple neurons fire for all input locations showing K patterns of interest.

What is target pattern in this example?

Fukushima, 1980

Illustrative Example of CNN Approach



This alternating process continues between simple and complex cells through the last layer!

Fukushima, 1980

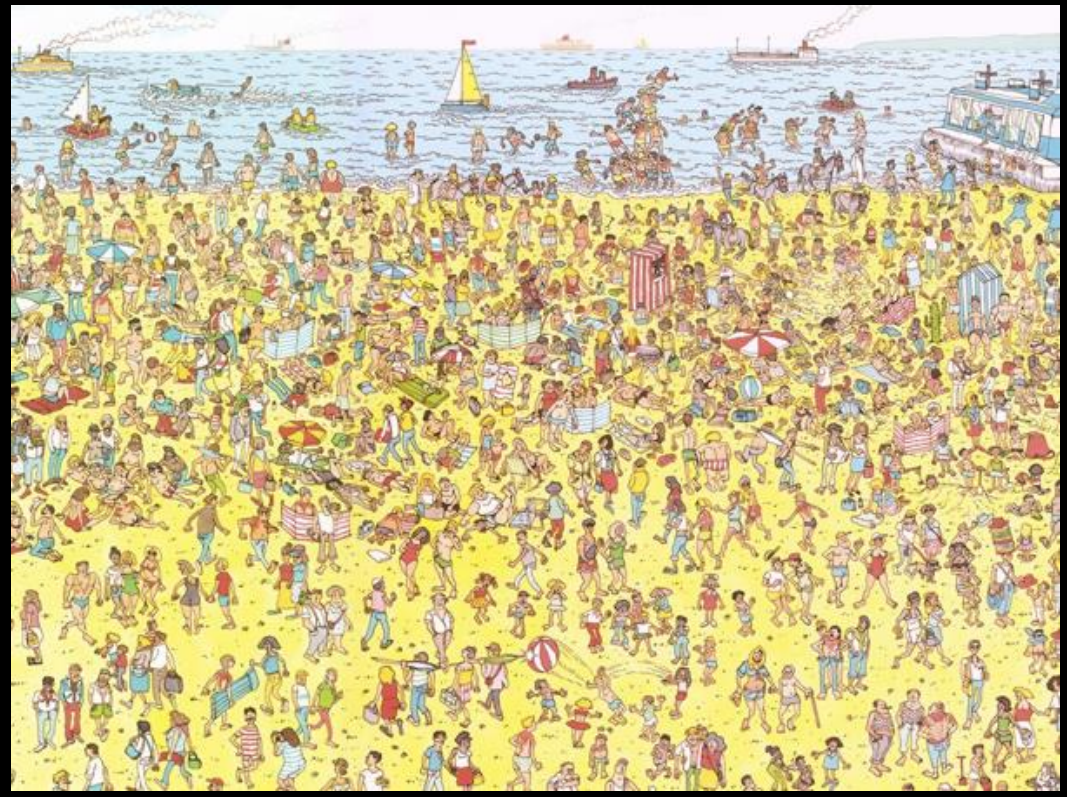
CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest;
  e.g., how might you find Waldo in the image?

Target Pattern:

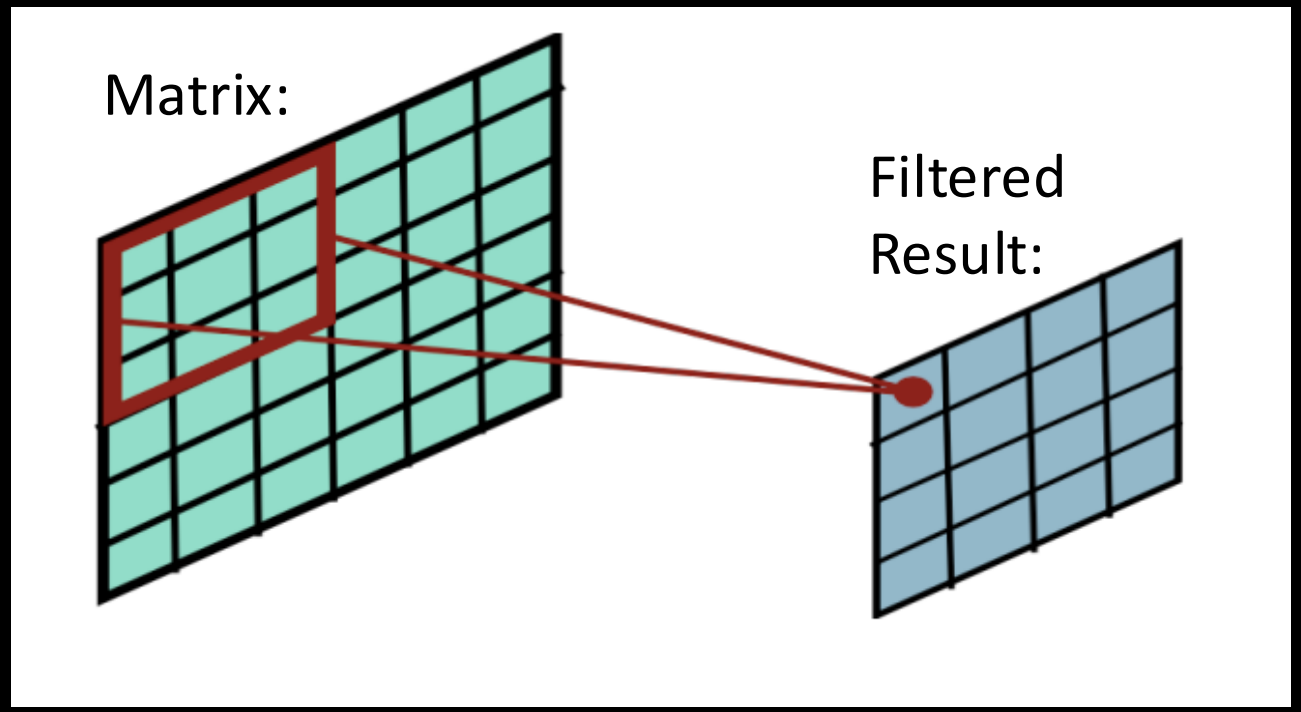


Input Image:



CNN: Convolutional Layer as a “Simple” Cell

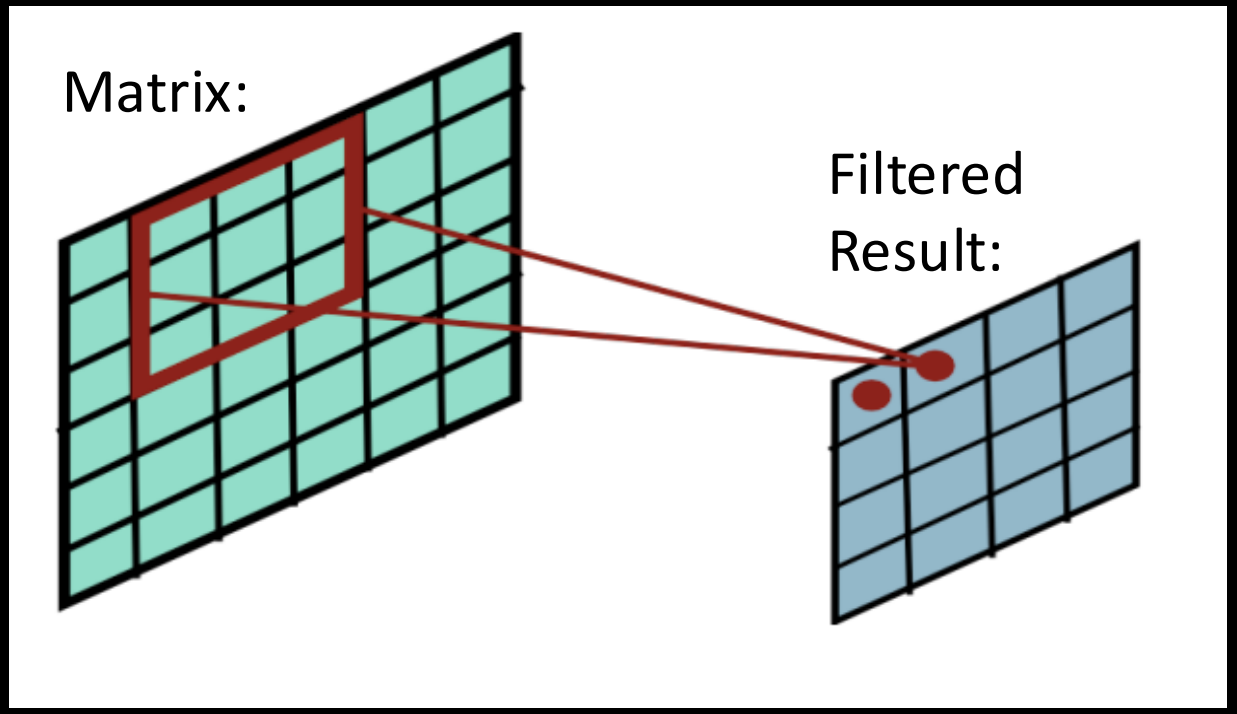
- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses



https://people.eecs.berkeley.edu/~jrs/189/lec/cnn.pdf

CNN: Convolutional Layer as a “Simple” Cell

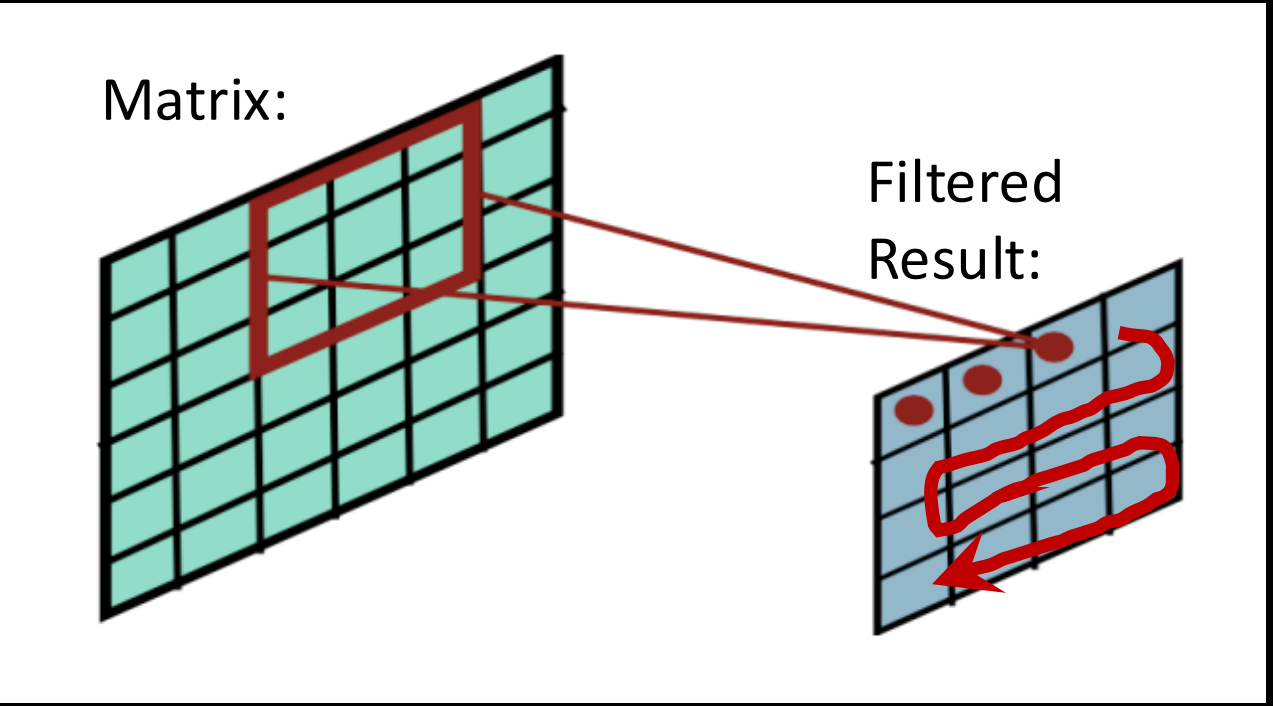
- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses



https://people.eecs.berkeley.edu/~jrs/189/lec/cnn.pdf

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses



Note: technically, this operation is “cross correlation” rather than “convolution”

https://people.eecs.berkeley.edu/~jrs/189/lec/cnn.pdf

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

What is the pattern we are looking for?

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

?		

Response = 1*1 + 1*0 + 1*1 + 0*0 + 1*1 + 1*0 + 0*1 + 0*1 + 0*0 + 0*0 + 1*1

Response = 4

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

What is the pattern we are looking for?

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

4		

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

4	?	

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

4	3	?

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

4	3	4
?		

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

4	3	4
2	?	

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

4	3	4
2	4	?

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

4	3	4
2	4	3
?		

CNN: Convolutional Layer as a “Simple” Cell

- Objective**:** fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

4	3	4
2	4	3
2	?	

CNN: Convolutional Layer as a “Simple” Cell

- Objective**:** fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Feature Map

4	3	4
2	4	3
2	3	?

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses; e.g.,

Filter

1	0	1
0	1	0
1	0	1

Input

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

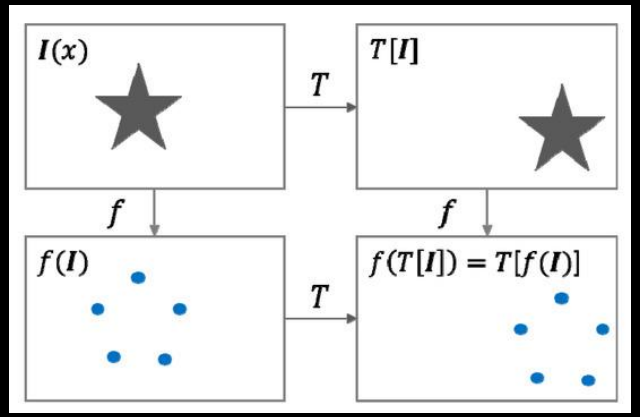
Feature Map

4	3	4
2	4	3
2	3	4

Where is the pattern:
- present?
- absent?

CNN: Convolutional Layer as a “Simple” Cell

- Objective: fire for all input locations showing a pattern of interest
- Approach: slide filter over matrix to find large matrix multiplication responses;
  - Compute a function with local neighborhoods in the matrix
  - The filter specifies the function for how to combine neighbors’ values
- Training goal: learn filter’s values (target patterns)
- Equivariant operation: shifting the input shifts the feature map in the same way, since the same filter gets applied! (i.e., no translation invariance yet)

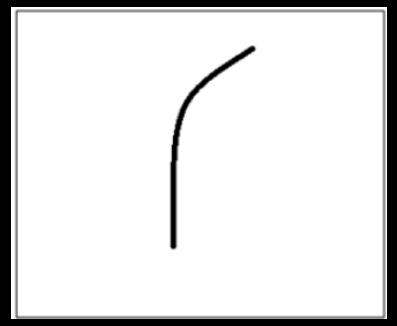


https://www.mdpi.com/2076-3417/10/9/3161

Many Types of Filters: What Do They Do?

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

https://adeshpande3.github.io/A-Beginner%27s-Guide-To-Understanding-Convolutional-Neural-Networks/

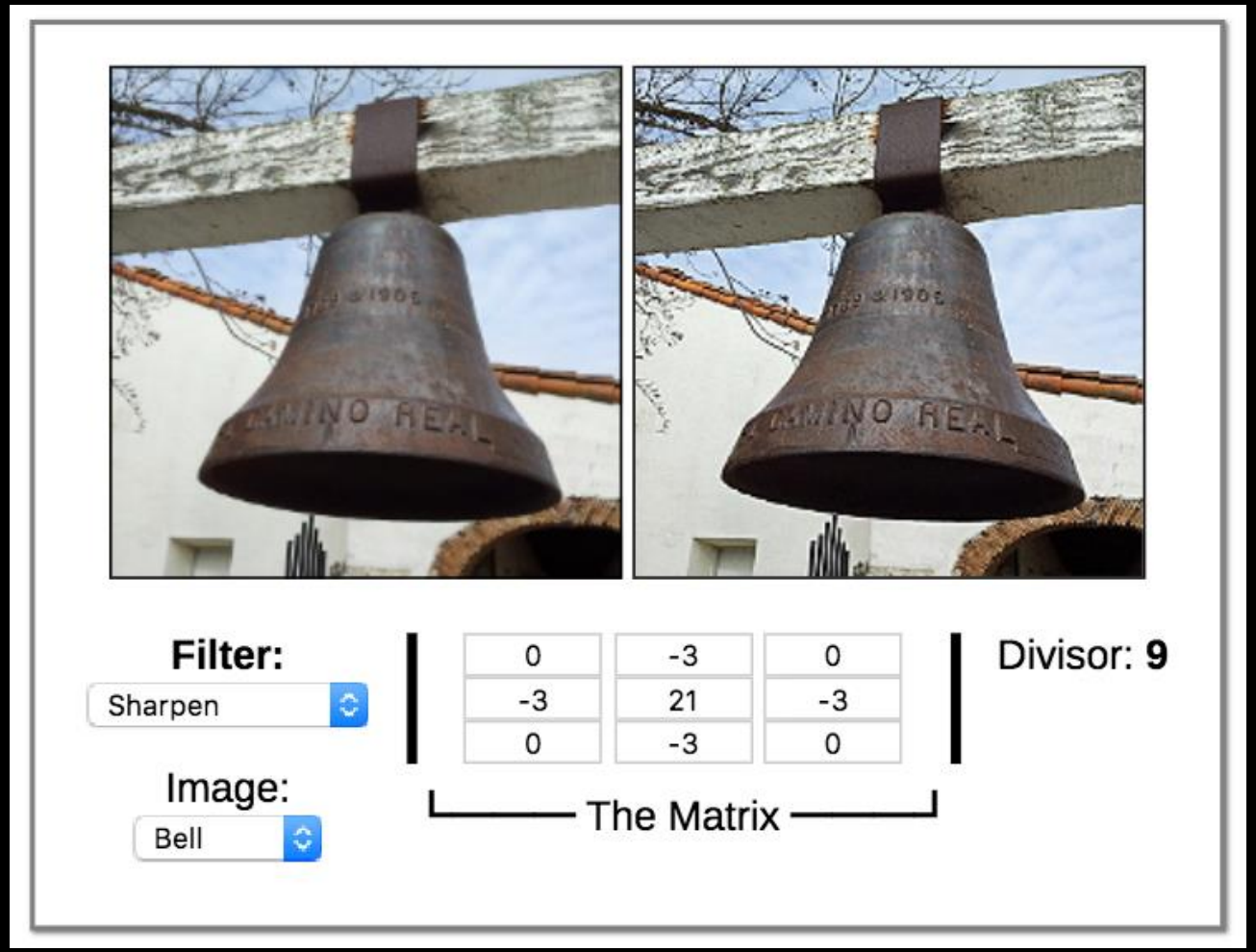


Filter | Feature Map

Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	

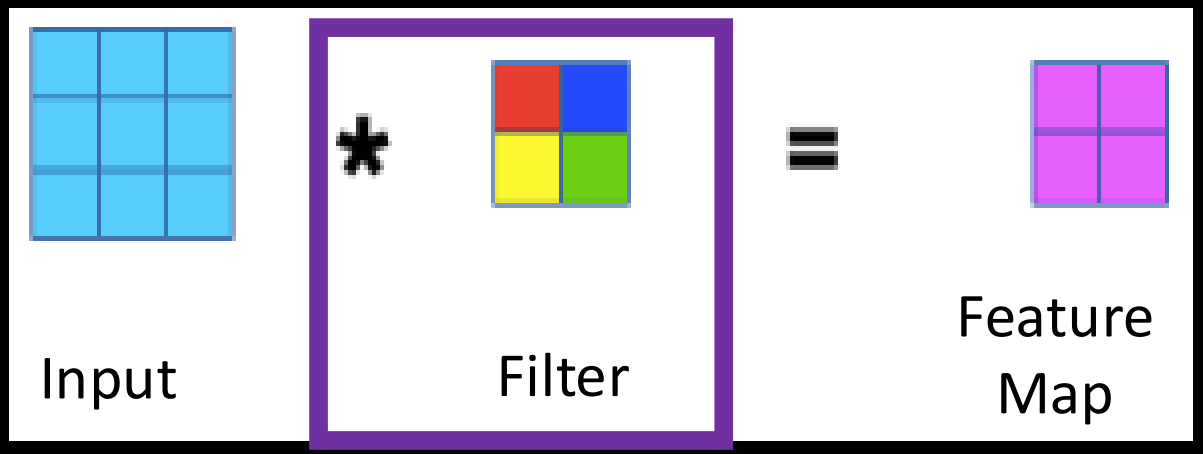
https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/

Many Types of Filters: What Do They Do?

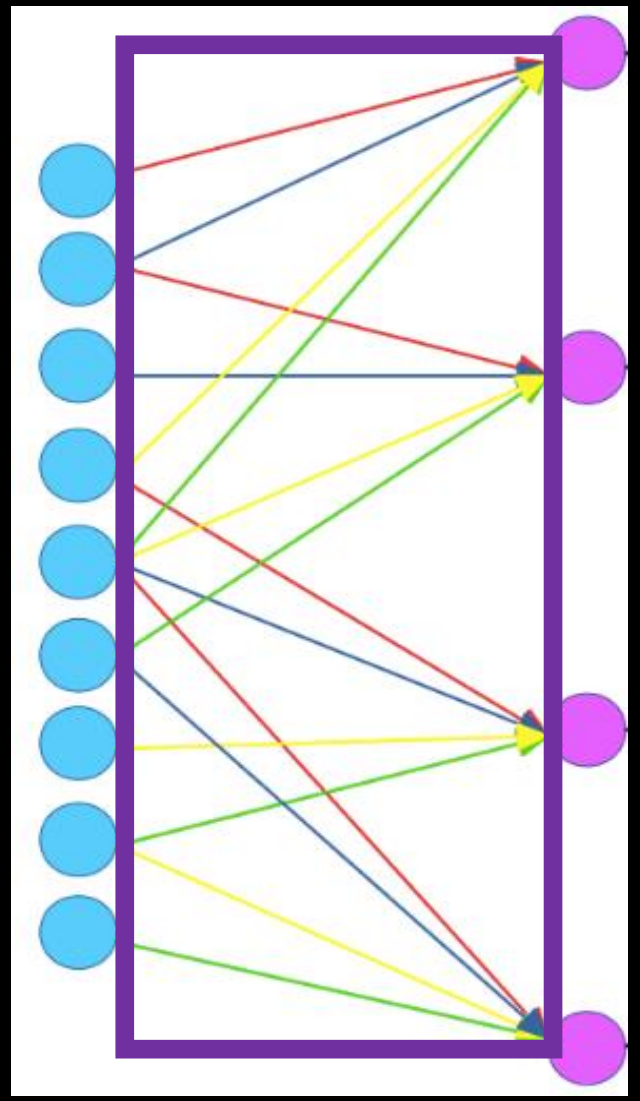


Demo: http://beej.us/blog/data/convolution-image-processing/

Convolution Interpreted within a NN



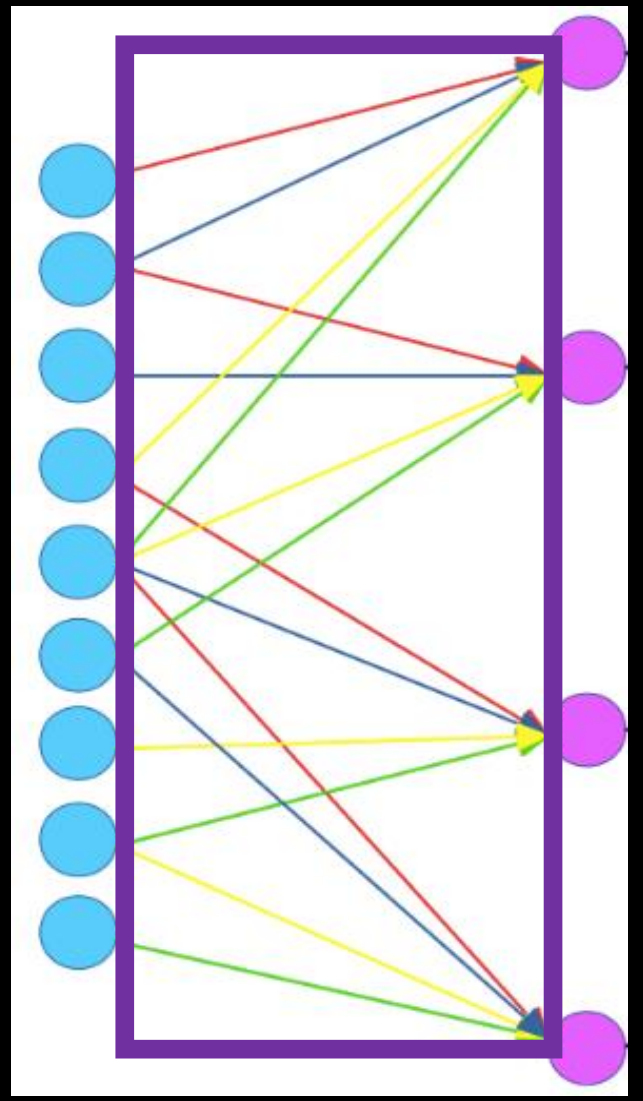




https://www.jefkine.com/general/2016/09/05/backpropagation-in-convolutional-neural-networks/

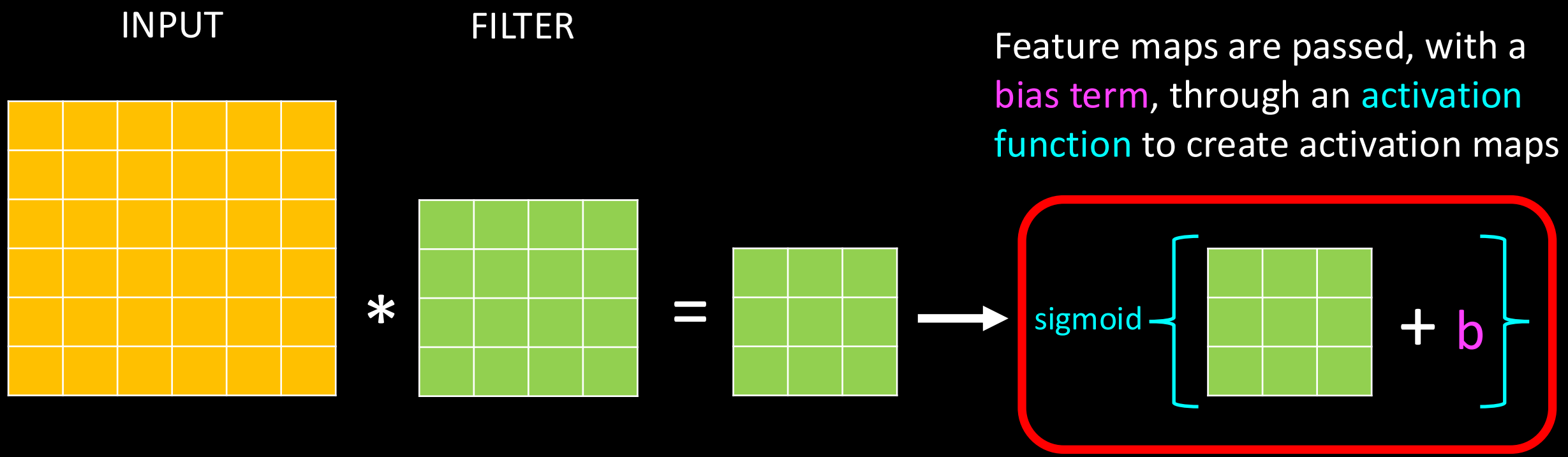
Convolutional vs Fully Connected Layers

- Number of model weights
  - How many weights must be learned for this convolutional layer?
    - 4 (red, blue, yellow, and green values)
  - How many weights would need to be learned if instead using a fully connected layer?
    - 36 (9 turquoise nodes x 4 magenta nodes)
- Number of inputs to neurons
  - How many inputs go to each neuron in the convolutional layer?
    - 4
  - How many inputs would go to each neuron if instead using a fully connected layer?
    - 9

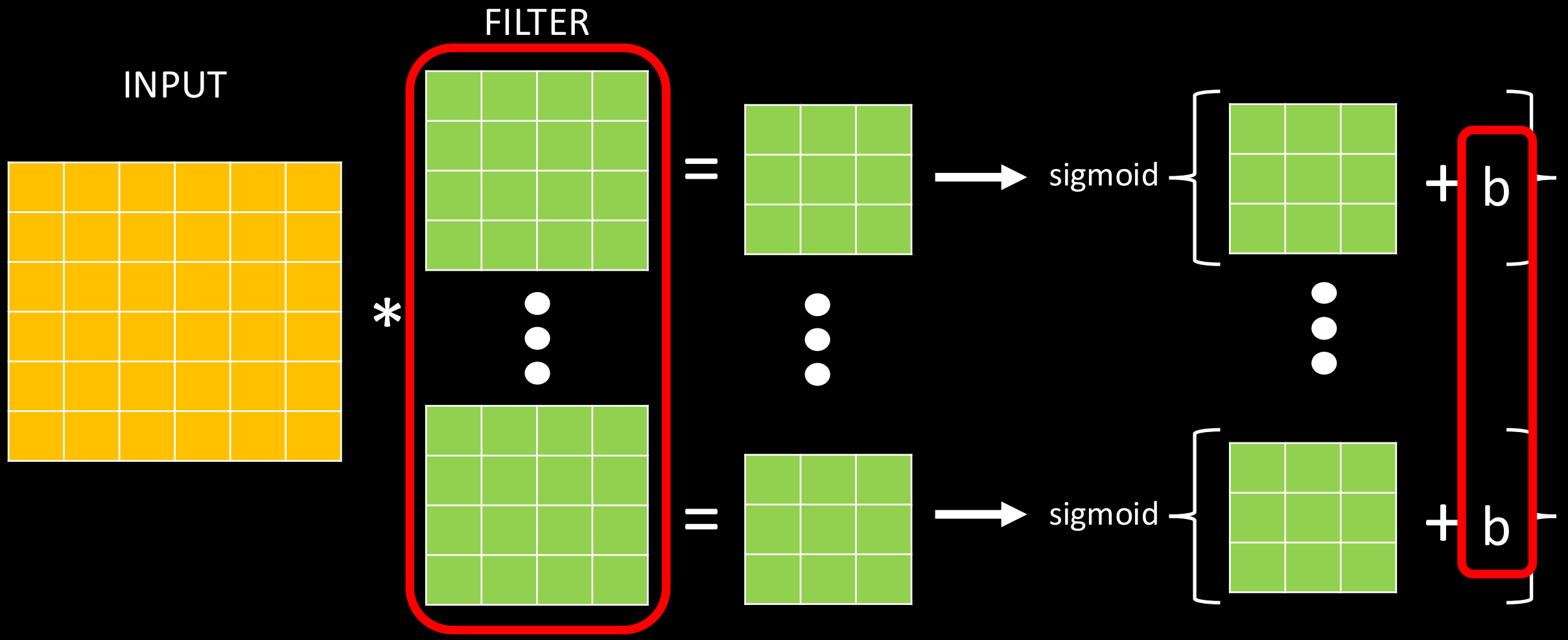


https://www.jefkine.com/general/2016/09/05/backpropagation-in-convolutional-neural-networks/

Convolutional Layer: Full Implementation

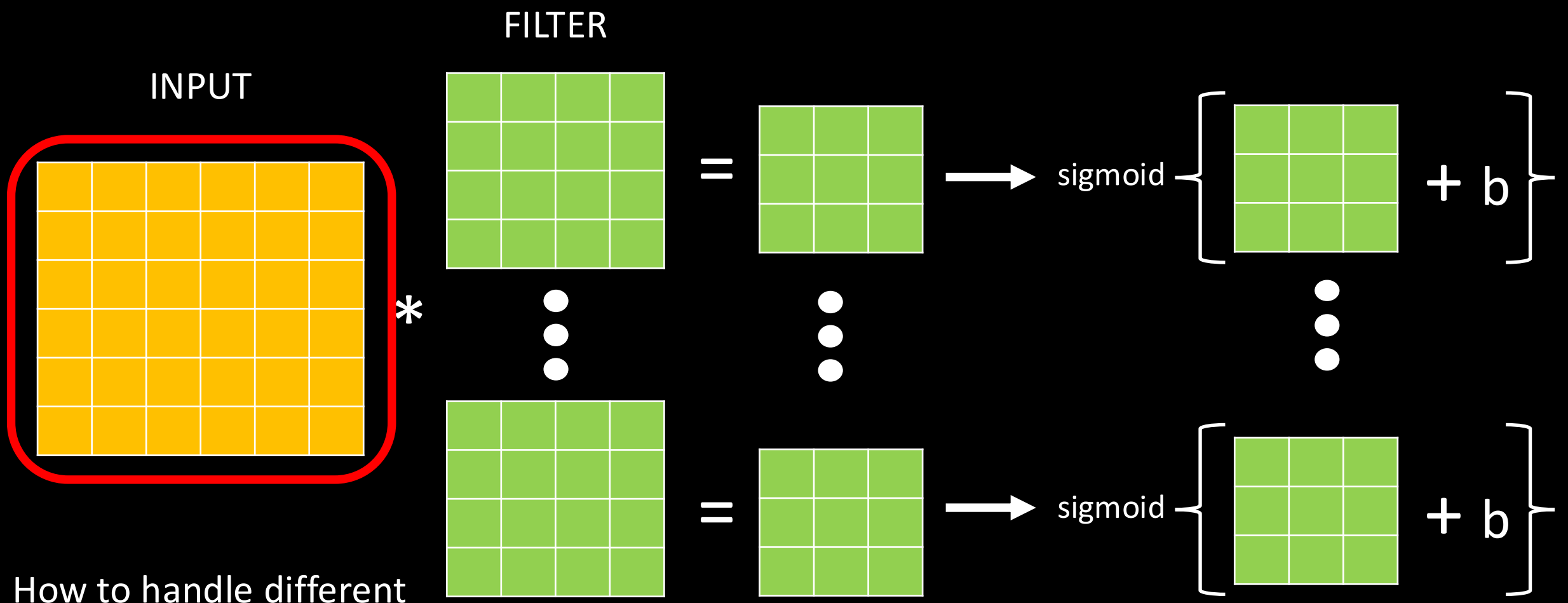


Convolutional Layer: Full Implementation



Can have multiple filters, each of any size with its own bias term;
all their values are learned during NN training

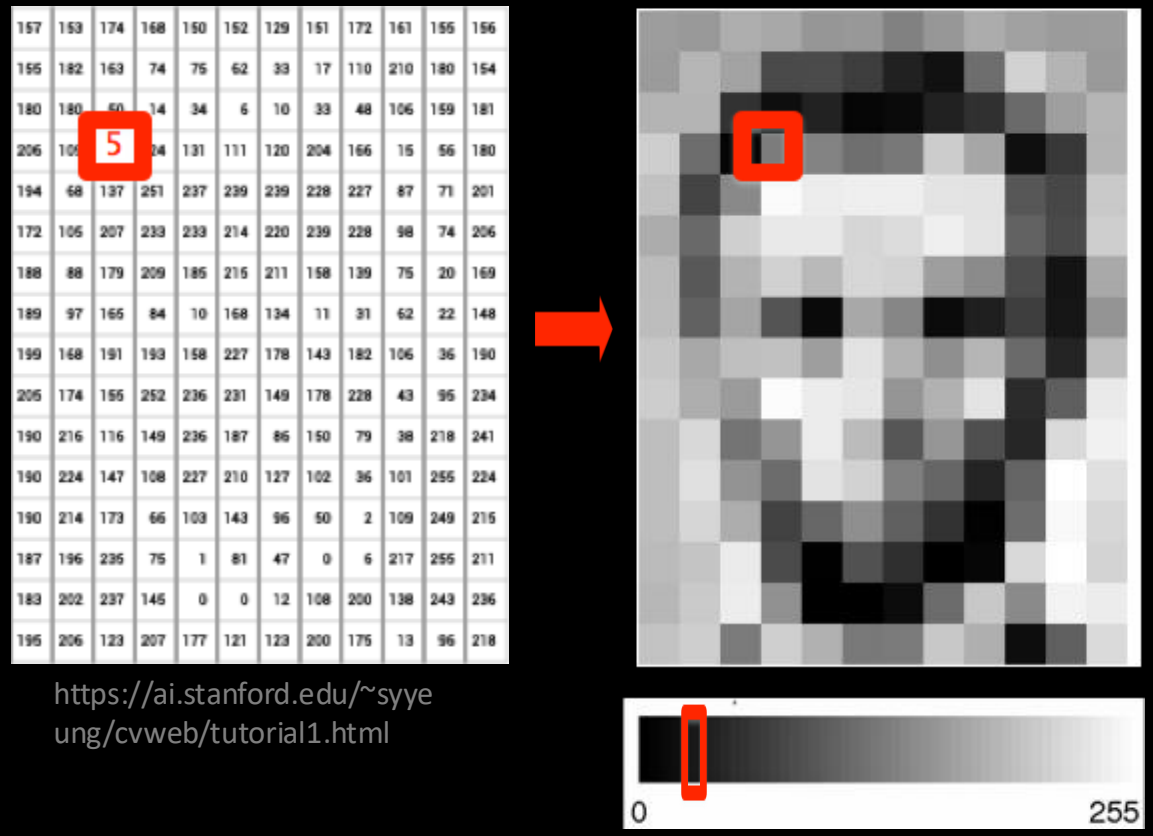
Convolutional Layer: Influence of Input



How to handle different numbers of input channels?

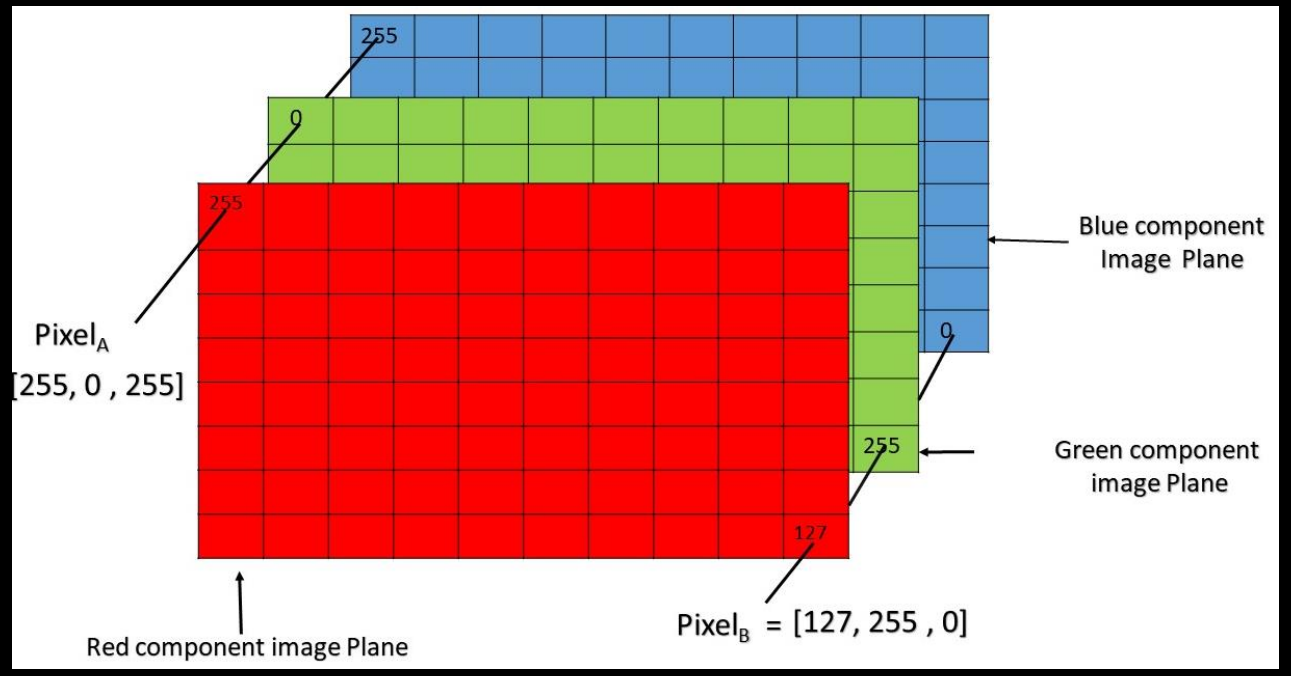
Convolutional Layer: Influence of Input

One Channel Input (8-bit greyscale image)



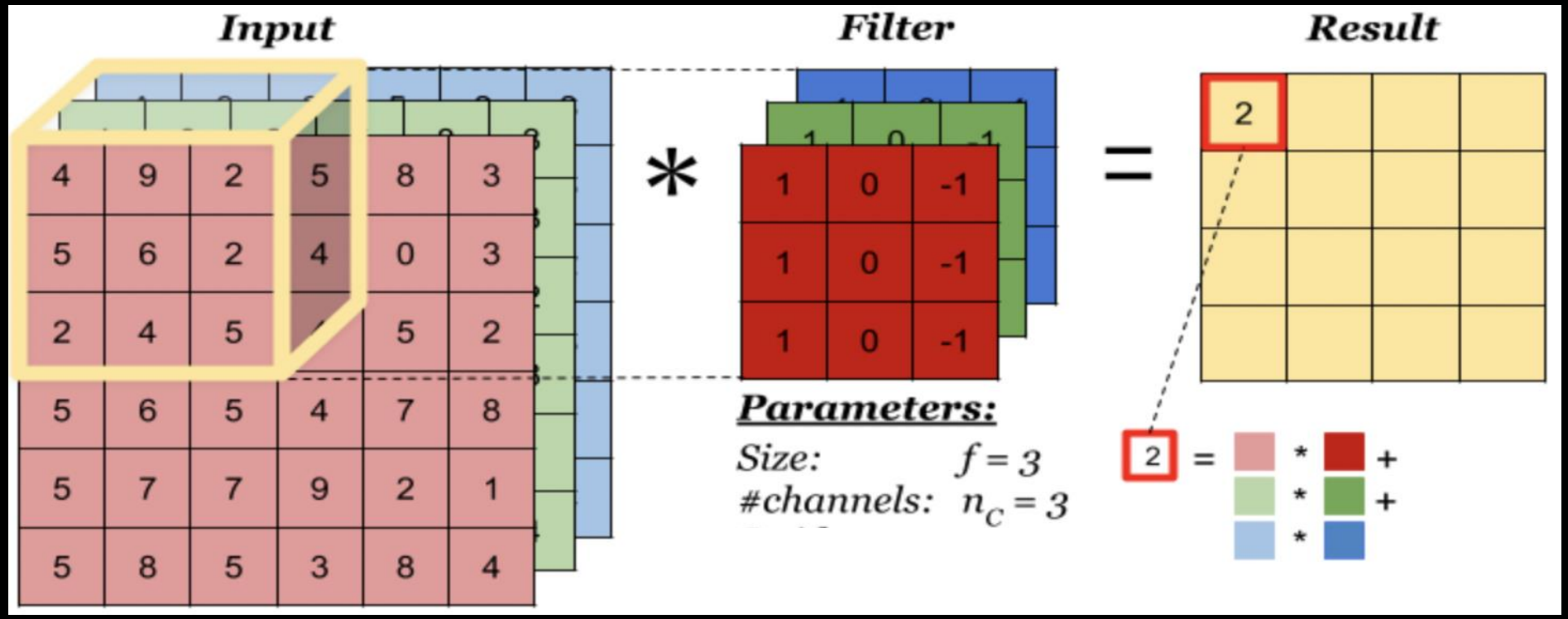
https://ai.stanford.edu/~syyeung/cvweb/tutorial1.html

Three Channel Input (RGB image)



https://www.geeksforgeeks.org/matlab-rgb-image-representation/

Convolutional Layer: Influence of Input



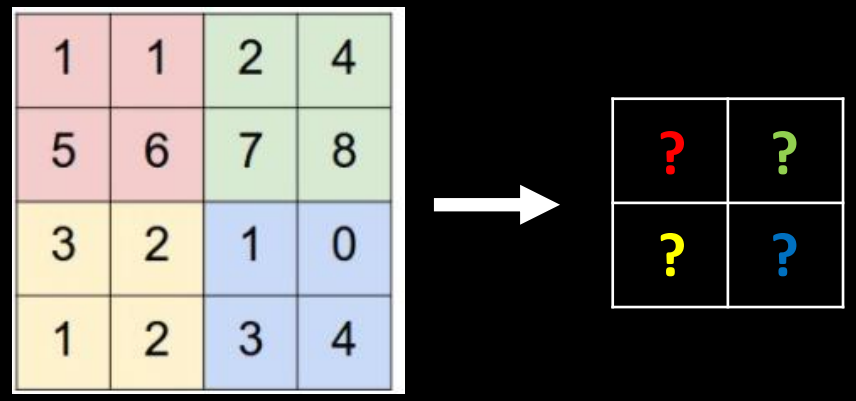
Number of channels in a filter must match that of the input, with each called a “kernel”

- how many filters are in this example?
- how many kernels are in this example?
- how many feature maps are created in this example?

https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/

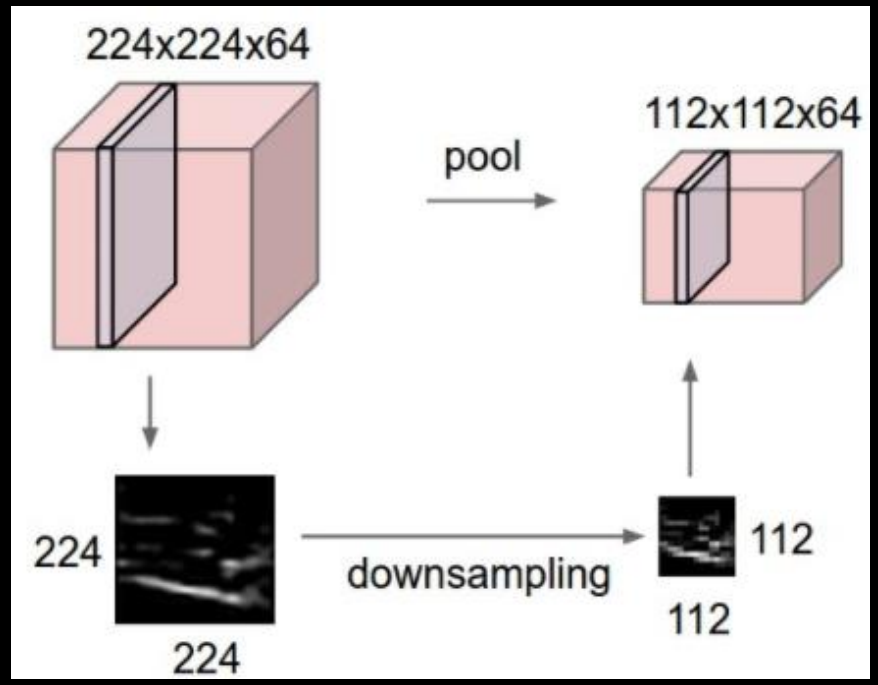
CNN: Pooling Layer as a “Complex” Cell

- Objective: fire when at least one part of a region shows a pattern of interest
- Approach: aggregates convolutional responses from a neighborhood; e.g., max pooling: outputs maximum value for partitioned input



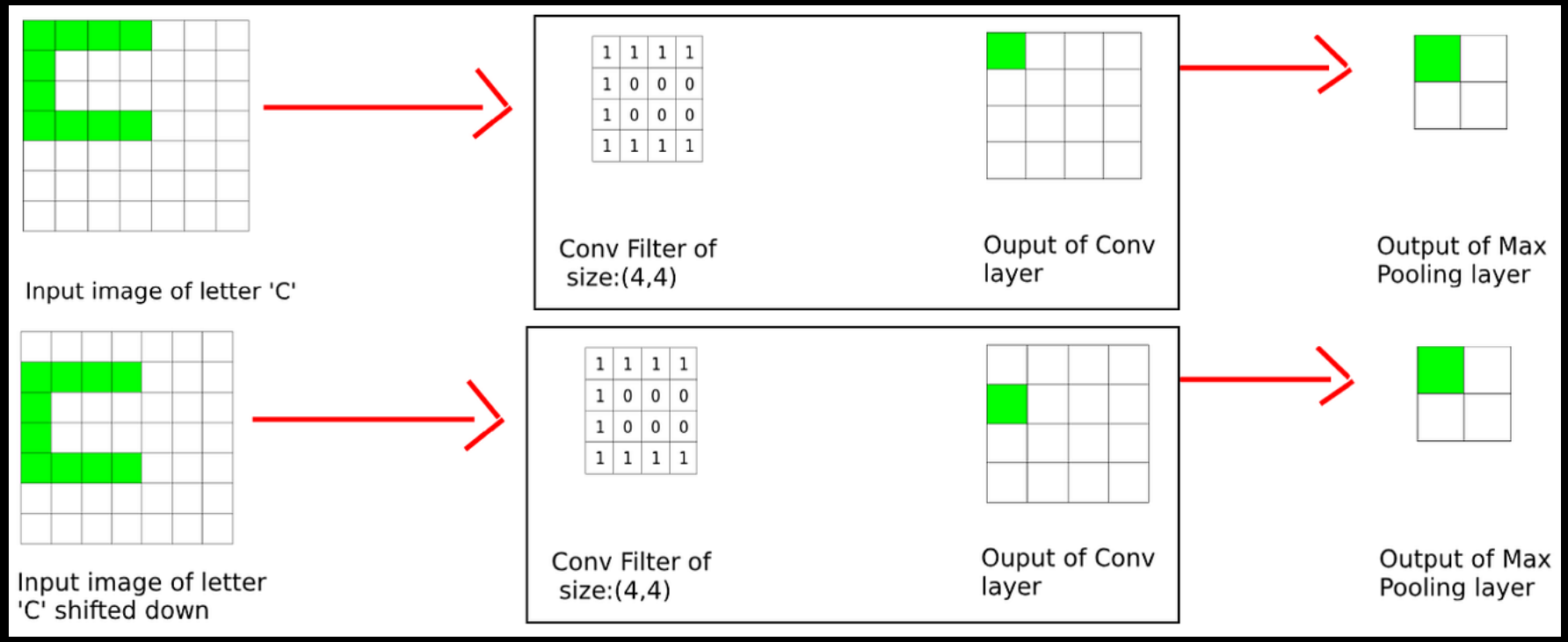
http://cs231n.github.io/convolutional-networks/#pool

- How do the outputs compare visually:



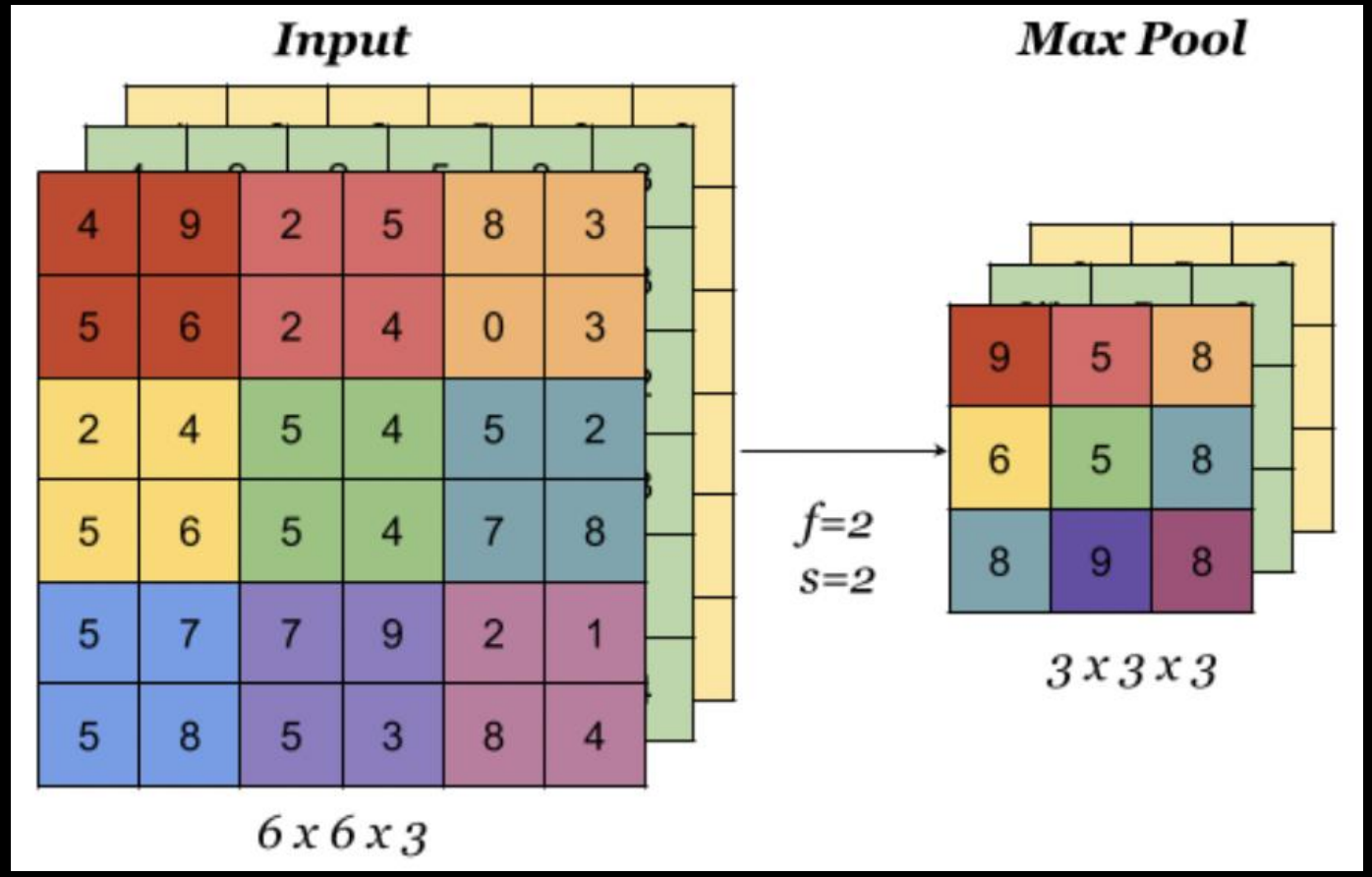
CNN: Pooling Layer as a “Complex” Cell

- Objective: fire when at least one part of a region shows a pattern of interest
- Approach: aggregates convolutional responses from a neighborhood
- Encodes translation invariance: shifting the input still results in the same output; e.g.,



https://divsoni2012.medium.com/translation-invariance-in-convolutional-neural-networks-61d9b6fa03df

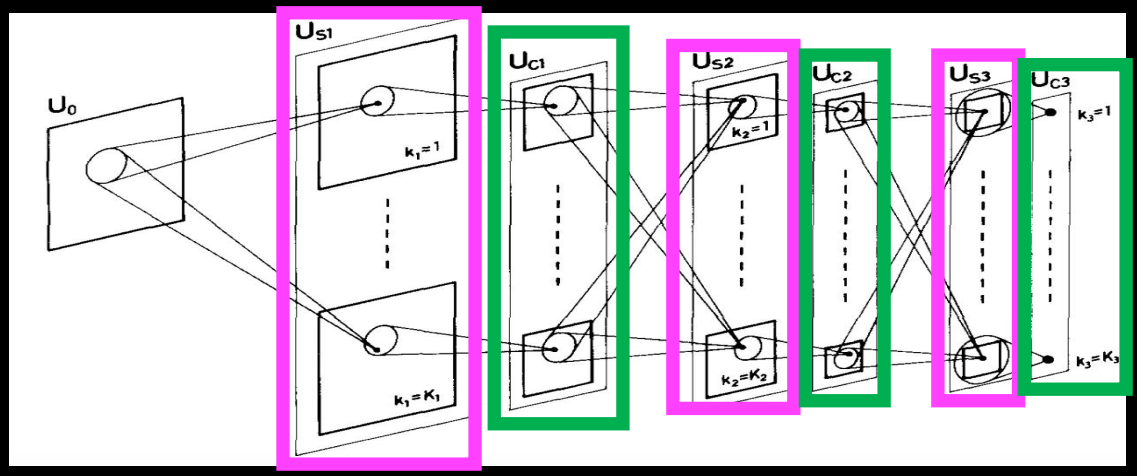
Pooling Layer: Influence of Input



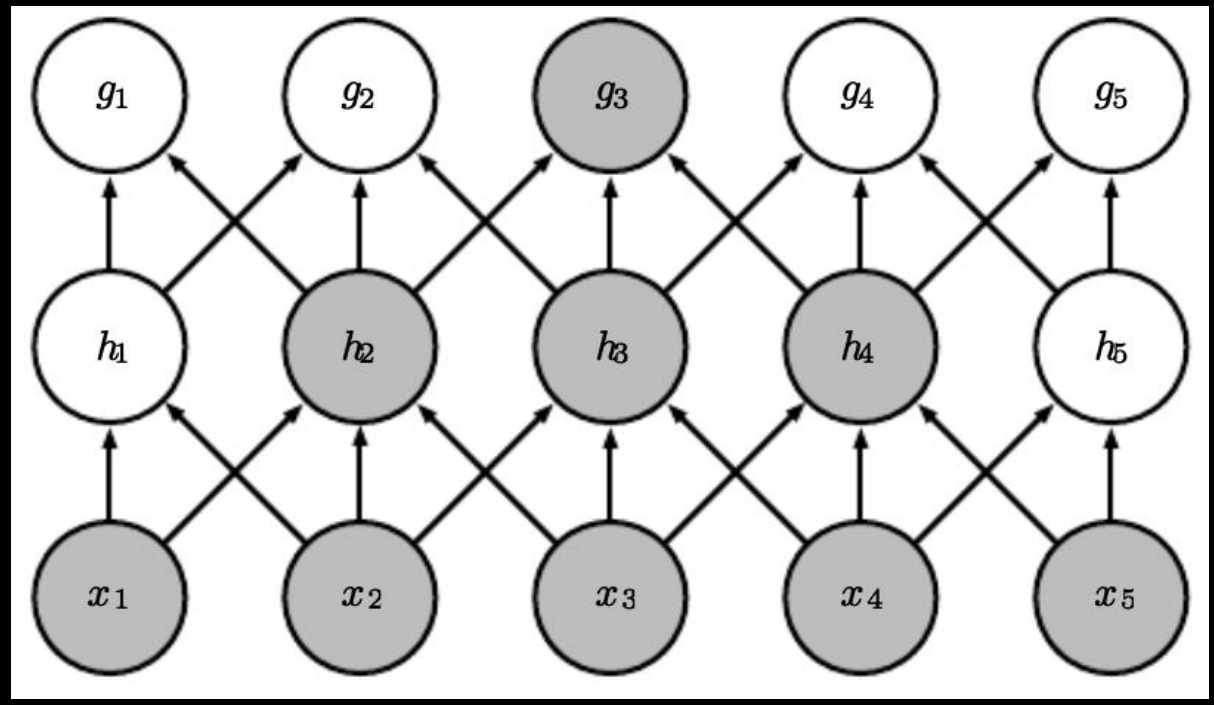
Pooling is applied to each input channel separately

https://indoml.com/2018/03/07/student-notes-convolutional-neural-networks-cnn-introduction/

CNN: Stacking Convolutional and Pooling Layers



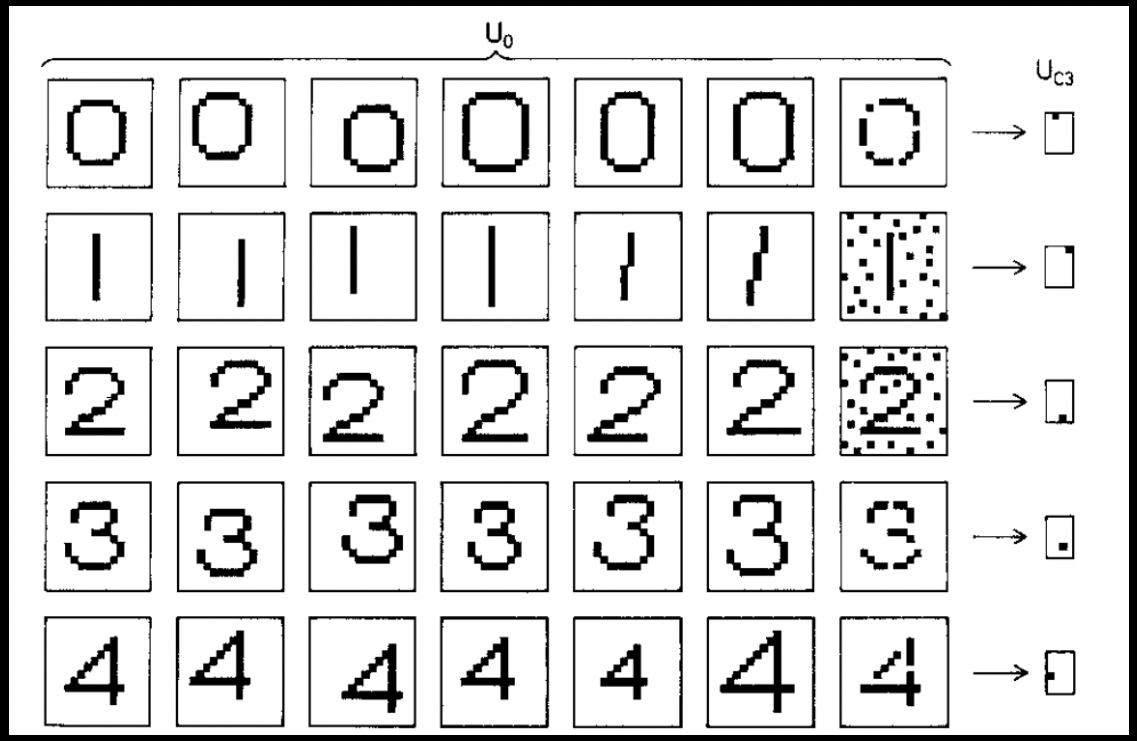
Fukushima, 1980



https://www.deeplearningbook.org/contents/convnets.html

Deeper layers identify patterns in larger regions of the input space, ultimately leading to greater translation invariance!

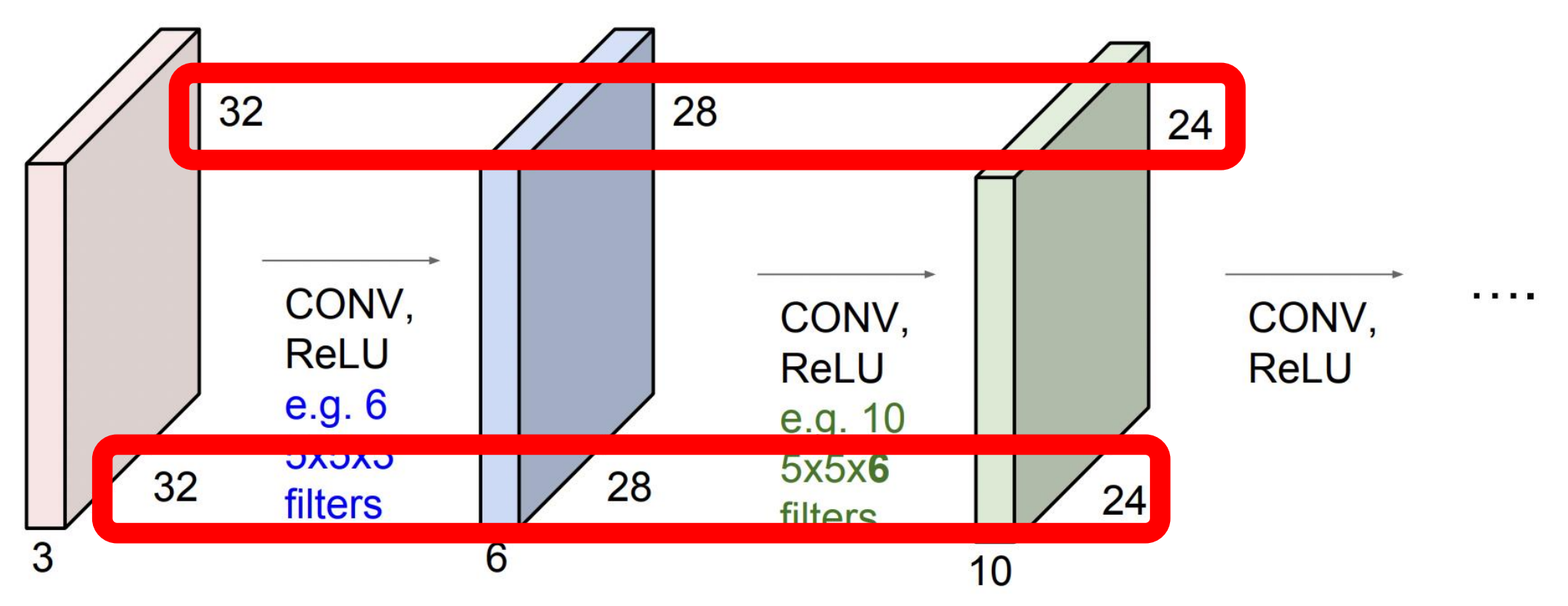
Experiments: Kunihiko's Neocognitron Worked!



Robust to inputs shifts, shape/size distortions, and noise!

Additional Design Elements (1 of 2)

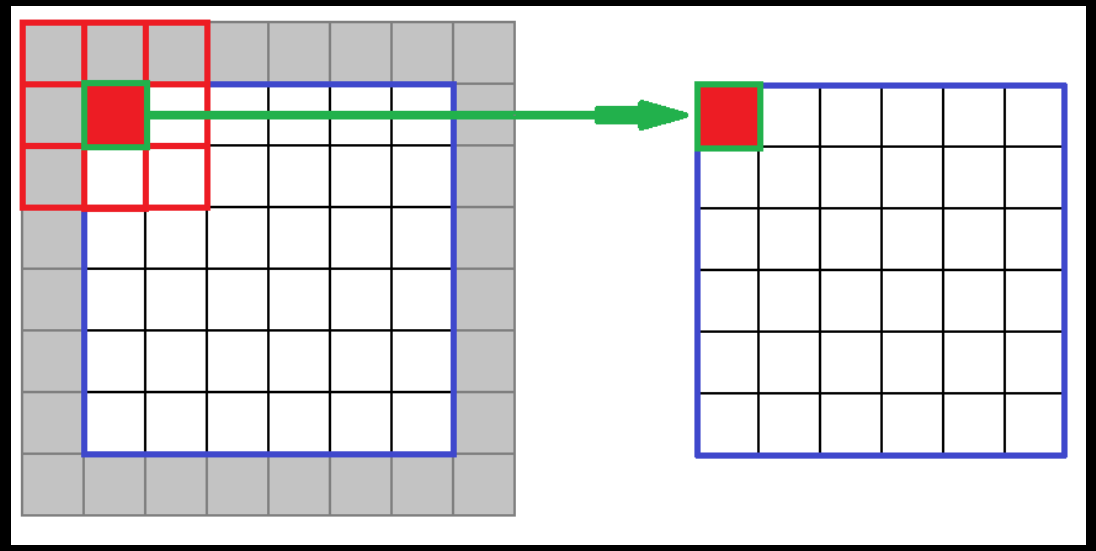
Why do the dimensions shrink with each convolutional layer?



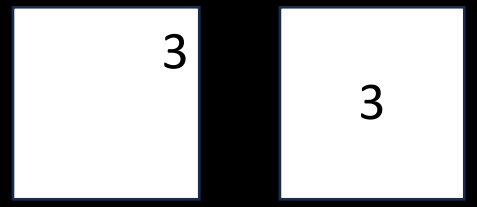
Information is lost around boundary of the input!

Additional Design Elements (1 of 2): Padding

- Idea: control output dimensions by padding the input
  - If using a 5x5 filter, how many layers are added to the boundary to retain original dimensions?

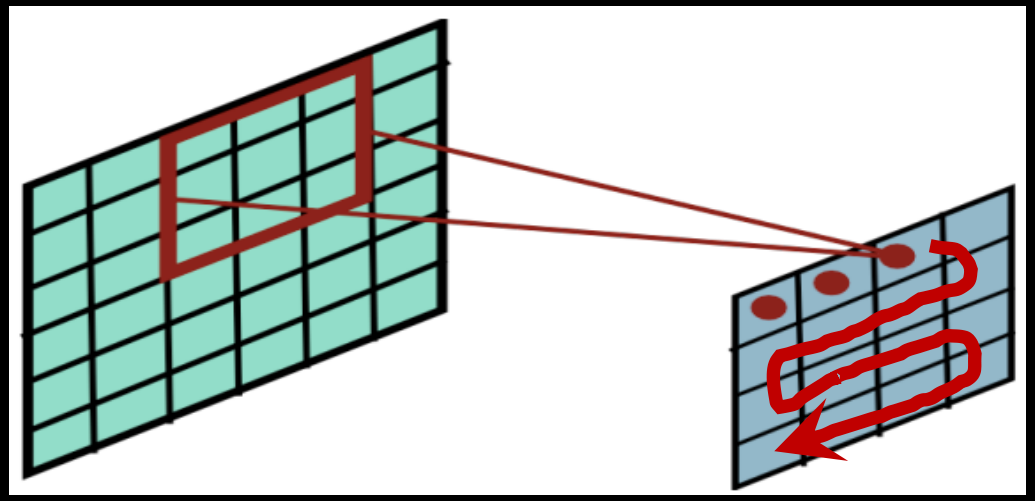


- Key considerations: is the boundary informative/valuable?



https://software.intel.com/en-us/node/586159

Additional Design Elements (2 of 2)



https://people.eecs.berkeley.edu/~jrs/189/lec/cnn.pdf

How many multiplications are needed for a convolutional layer with one 3x3 kernel on an RGB image of dimension 1280 x 720? (assume padding to original dimensions)

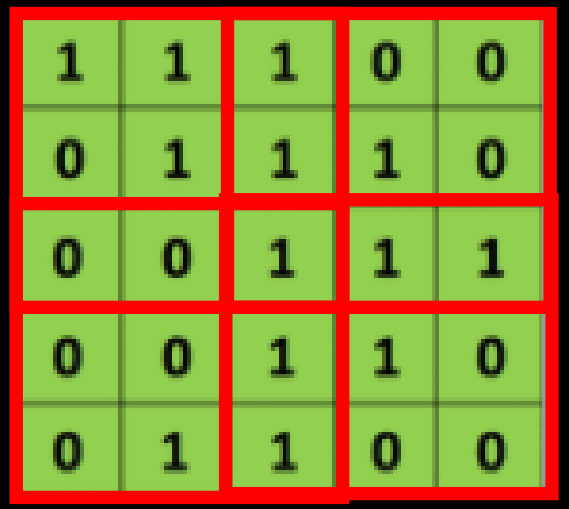
(3x3) x (1280 x 720) x 3 = 24,883,200

Computationally expensive!

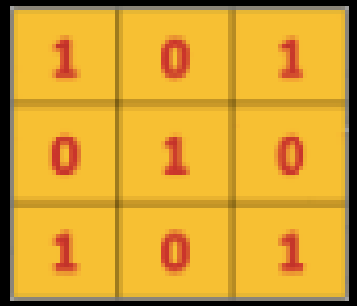
Additional Design Elements (2 of 2): Stride

- Idea: reduce how many steps are taken spatially
  - e.g., 2x2

Input



Filter



Feature Map

4	4
2	4

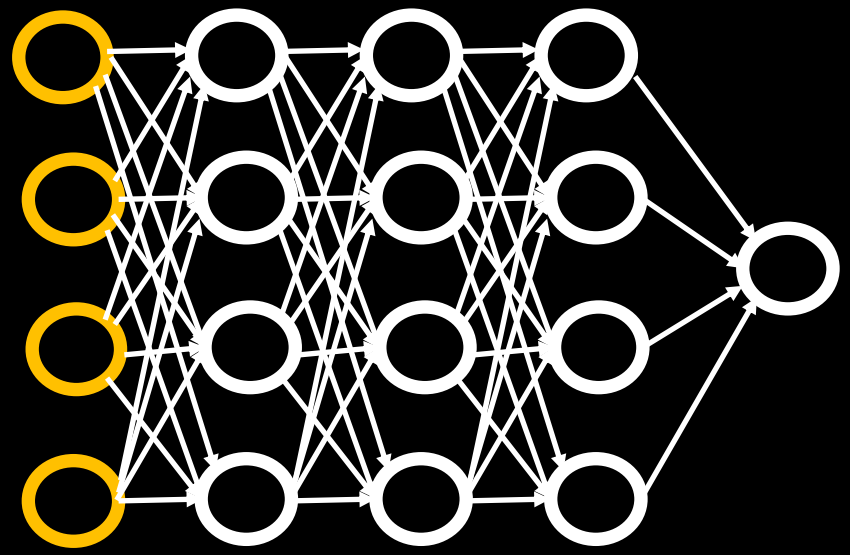
- Key considerations: how would downsampling impact recognition?

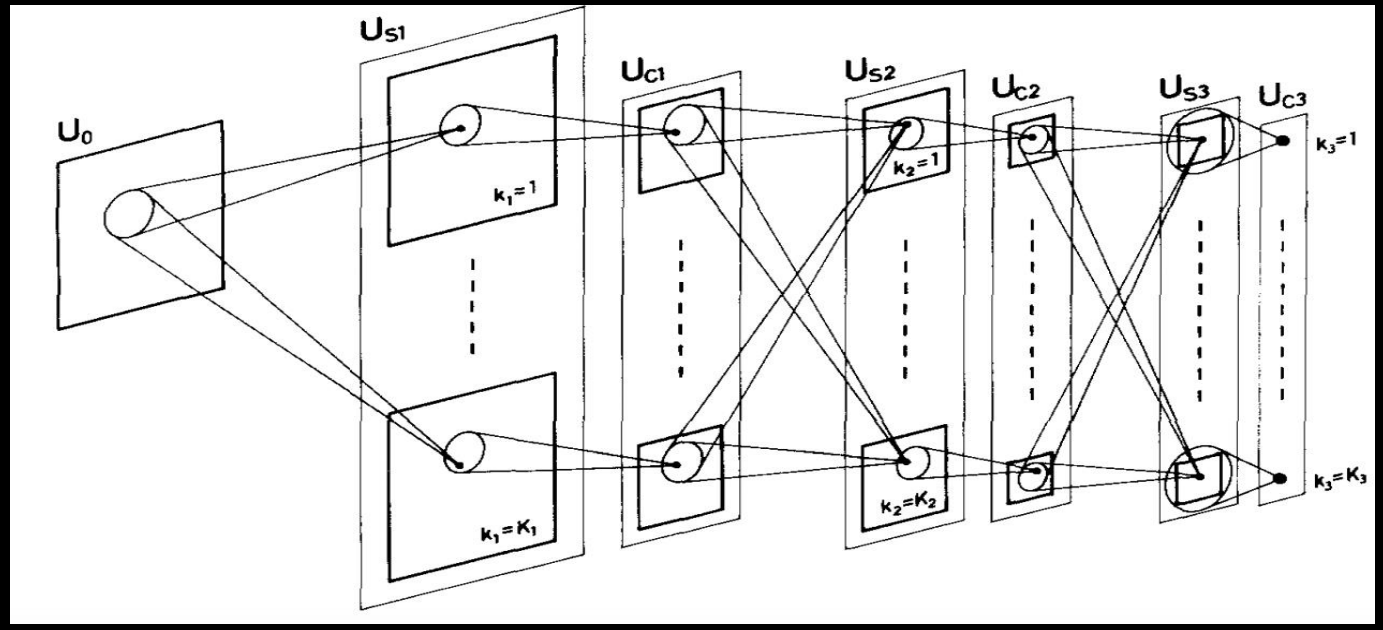
3

3

http://deeplearning.net/software/theano/tutorial/conv_arithmetic.html

Summary: Fully Connected Networks vs CNNs





Fukushima, 1980

Can learn translation invariance by learning similar feature detectors at many spatial locations (when given lots of training data)

CNNs avoid learning redundancy by introducing architectural inductive biases—shared weights tied to local receptive fields enable reusable feature detectors while spatial pooling builds invariance

Today's Topics

- Issue: input shifts cause fully connected networks to fail
- Idea: convolutional neural networks (CNNs) designed for translation invariance
- Implementation: training CNNs end-to-end with gradient descent
- Real-world success: LeNet-5 for check reading
- Programming tutorial: convolutional neural networks in PyTorch

Goal: Supervised Learning of CNN Parameters



Yann LeCun
1960 – Present
2018 Turing Award winner for contributions to deep learning
Professor, Chief AI Scientist (Meta)

Backpropagation Applied to Handwritten Zip Code Recognition

Y. LeCun
B. Boser
J. S. Denker
D. Henderson
R. E. Howard
W. Hubbard
L. D. Jackel
AT&T Bell Laboratories Holmdel, NJ 07733 USA

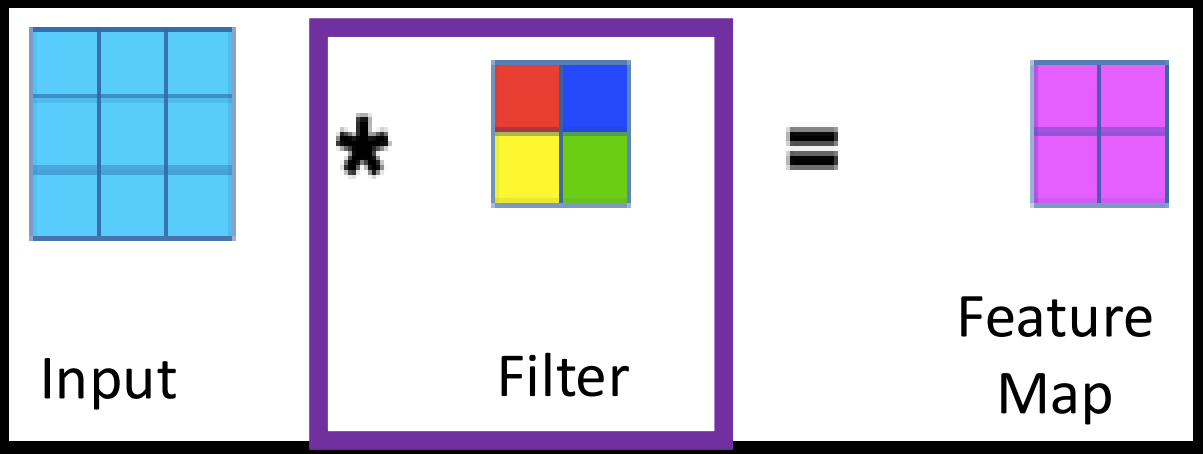
Neural computation, 1989

CNN Training: Gradient Based Learning

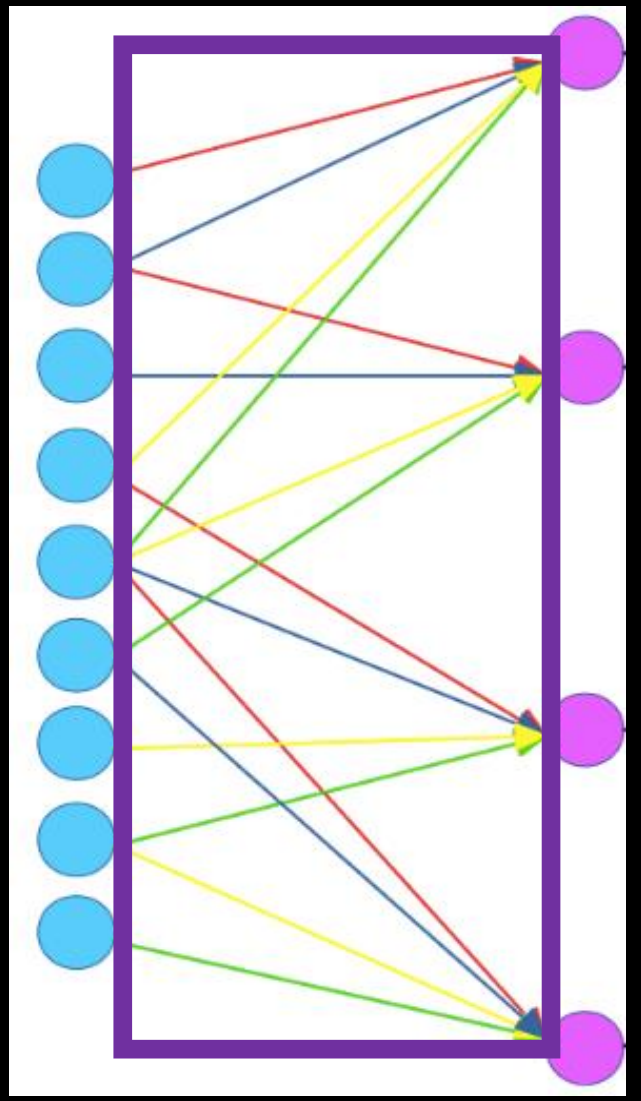
- Repeat until stopping criterion met:

1. Propagate training data through network to predict output
2. Measure error of predictions using a loss function
3. Calculate gradients of the loss function to determine how each model parameter contributed to the error; weight sharing accounted for by summing partial derivative values for all connections that share the same parameter
4. Update each parameter using calculated gradients

CNN Training: Weight Sharing







Partial derivative values for all connections that share the same parameter get summed

https://www.jefkine.com/general/2016/09/05/backpropagation-in-convolutional-neural-networks/

CNN Training: Gradient Based Learning

- Repeat until stopping criterion met:

    1. Propagate training data through network to predict output

    2. Measure error of predictions using a loss function

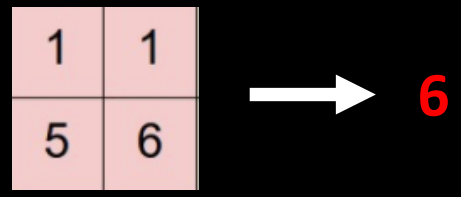
    3. Calculate gradients of the loss function to determine how each model parameter contributed to the error; weight sharing accounted for by summing partial derivative values for all connections that share the same parameter

    4. Update each parameter using calculated gradients

How to handle subsampling strategies (pooling layers, stride)?

Subsampling Strategies: Pooling and Stride

- Approach: only inputs that contribute to an output are blamed and so receive gradients for parameter updating; all other paths get 0 gradient
  - e.g., max pooling: gradient flows only to the input that was the max with other paths getting 0 gradient; since no learning happens, the gradient just passes through to the earlier layer



Today's Topics

- Issue: input shifts cause fully connected networks to fail
- Idea: convolutional neural networks (CNNs) designed for translation invariance
- Implementation: training CNNs end-to-end with gradient descent
- Real-world success: LeNet-5 for check reading
- Programming tutorial: convolutional neural networks in PyTorch

Breakthrough: Success in Real-World Application

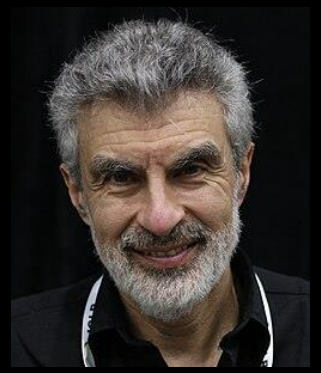
Proceedings of the IEEE, 1998

Gradient-Based Learning Applied to Document Recognition

Yann LeCun Léon Bottou Yoshua Bengio and Patrick Haffner



Yann LeCun
1960 – Present
2018 Turing Award winner for deep learning contributions
Professor, Chief AI Scientist (Meta)



Yoshua Bengio
1964 – Present
2018 Turing Award winner for deep learning contributions
Professor, Co-President of LawZero

Breakthrough: Success in Real-World Application

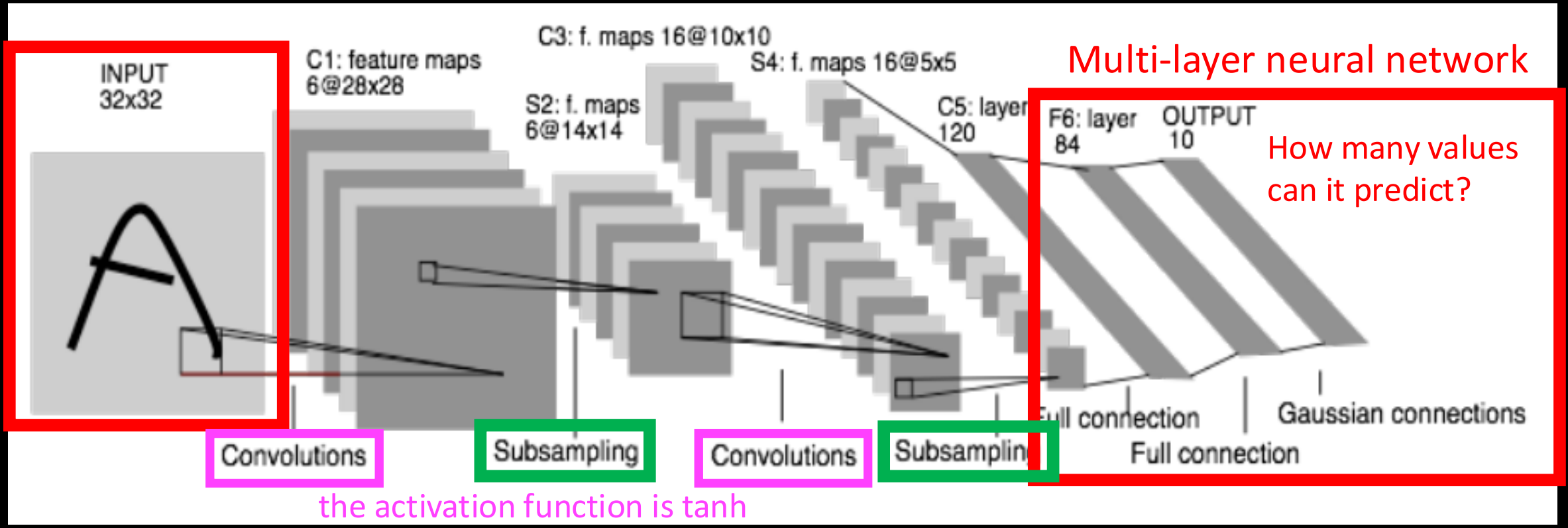
Proceedings of the IEEE, 1998

Gradient-Based Learning Applied to Document Recognition

Yann LeCun Léon Bottou Yoshua Bengio and Patrick Haffner

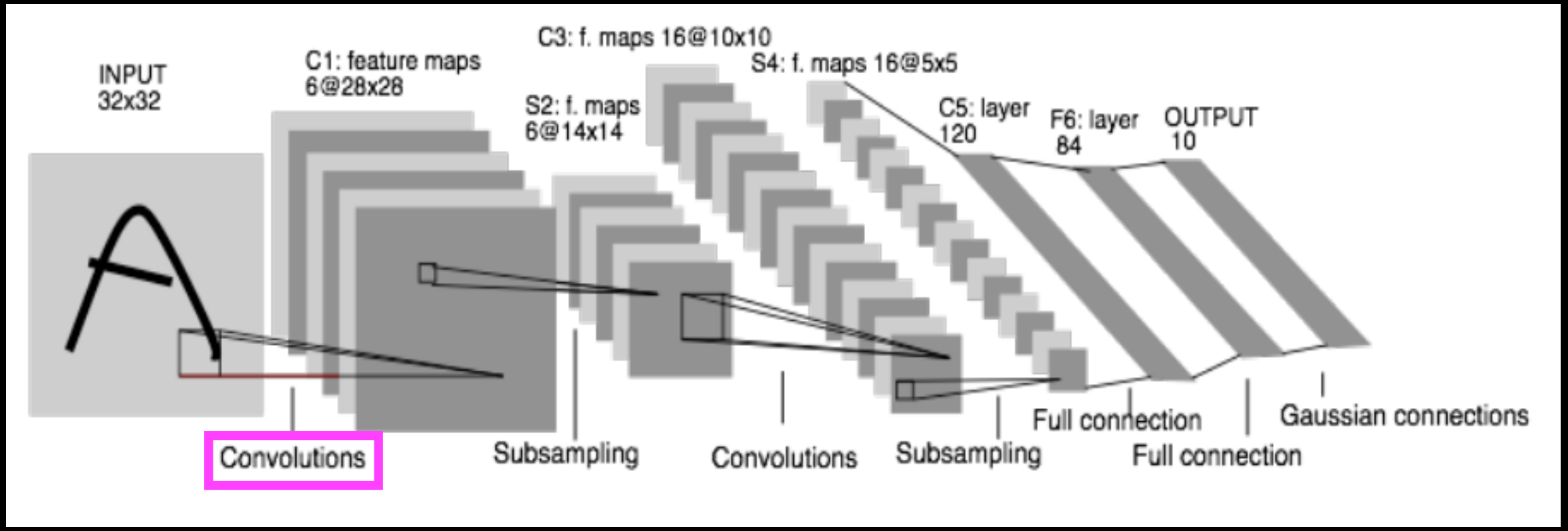
"It is reading millions of checks per month in several banks across the United States."

LeNet-5: Architecture (like Neocognitron, has alternating convolutional layers and pooling layers)



LeCun et al. Gradient-based learning applied to document recognition. 1998

LeNet-5: Architecture (like Neocognitron, has alternating convolutional layers and pooling layers)



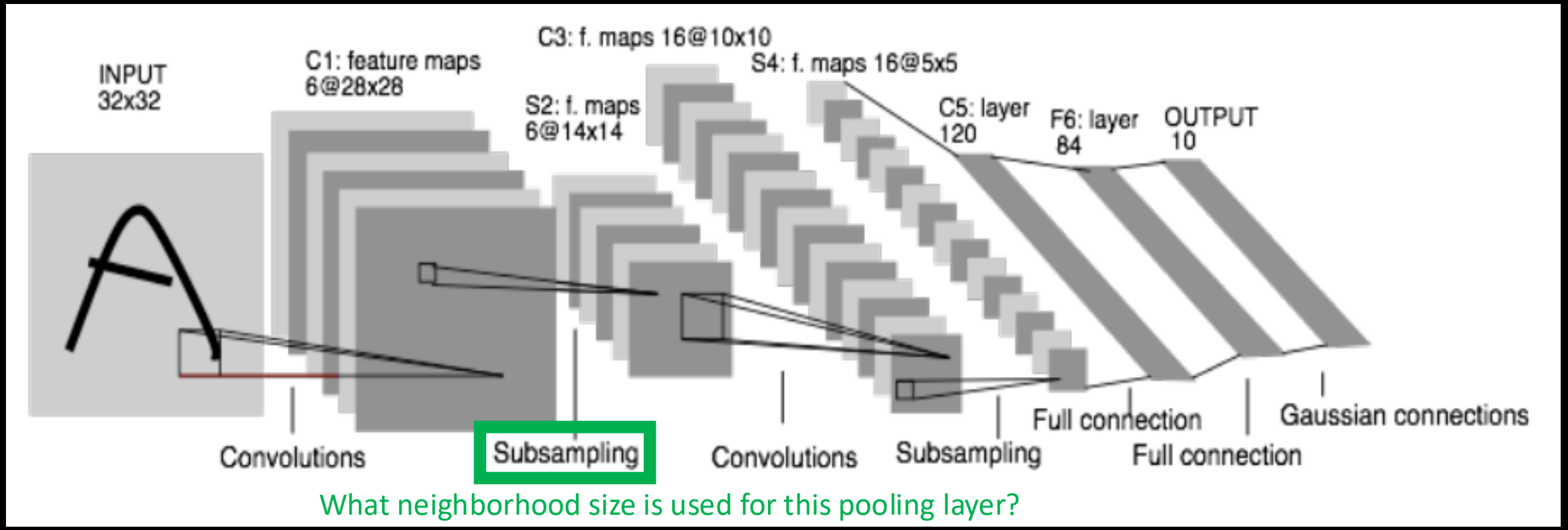
How many filters are between the input and hidden layer 1?

What are their sizes?

How many parameters are between the two layers? – 6 x 5 x 5 + 6 = 156

LeCun et al. Gradient-based learning applied to document recognition. 1998

LeNet-5: Architecture (like Neocognitron, has alternating convolutional layers and pooling layers)

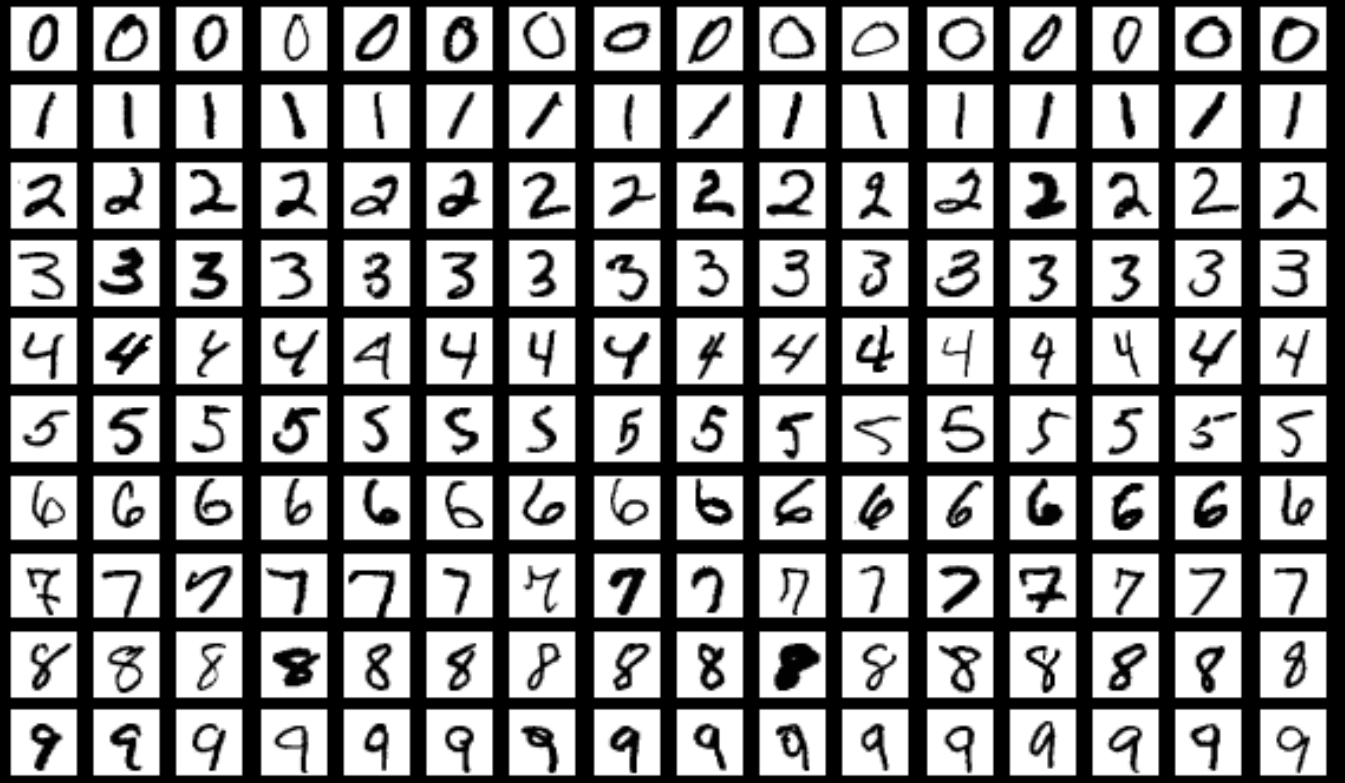


What neighborhood size is used for this pooling layer?

LeCun et al. Gradient-based learning applied to document recognition. 1998

Dataset for Model Development: MNIST

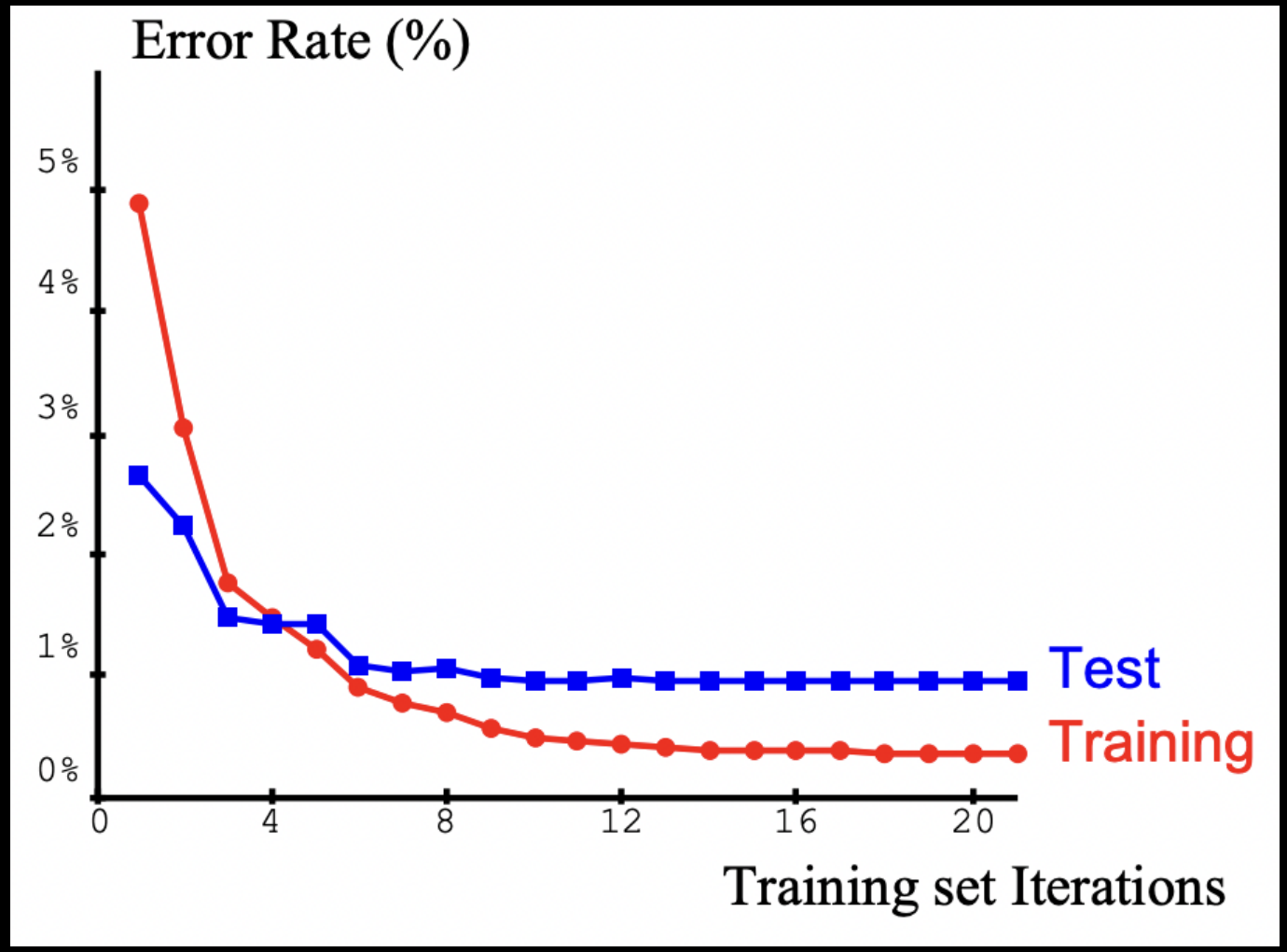
- Task classify digit as 0, 1, …, or 9
- Source: images collected by NIST from a total of 500 Census Bureau employees and high school students
- Dataset: 60,000 training and 10,000 test examples, pre-processed to be centered and same dimension; writers were different in the two sets
- Evaluation metric: accuracy (% correct)



https://commons.wikimedia.org/w/index.php?curid=64810040

LeNet-5 Analysis

How many epochs are needed for training to converge?



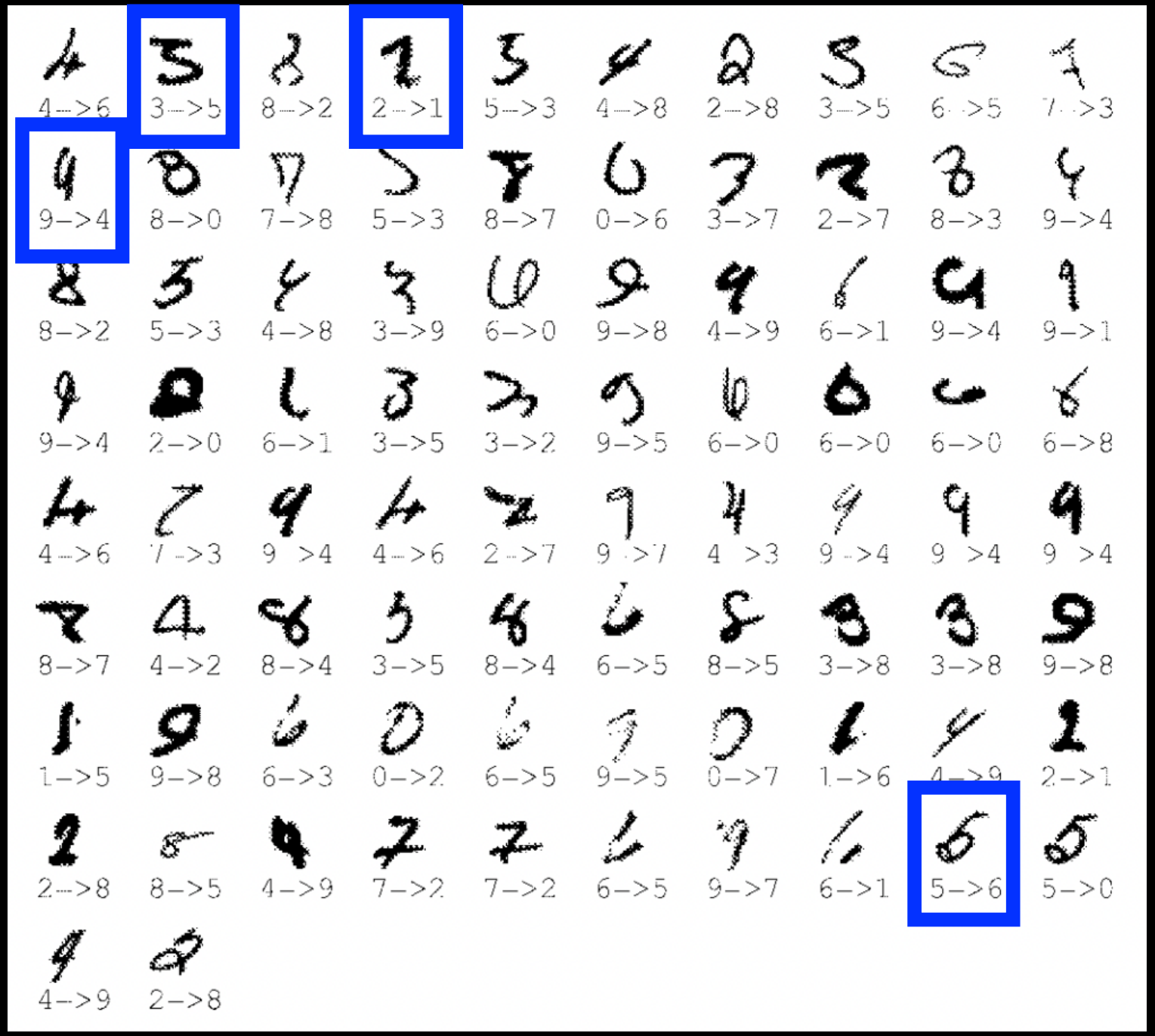
LeCun et al. Gradient-based learning applied to document recognition. 1998

LeNet-5 Analysis

All 82 mislabeled examples (correct answer on left, predicted answer on right):

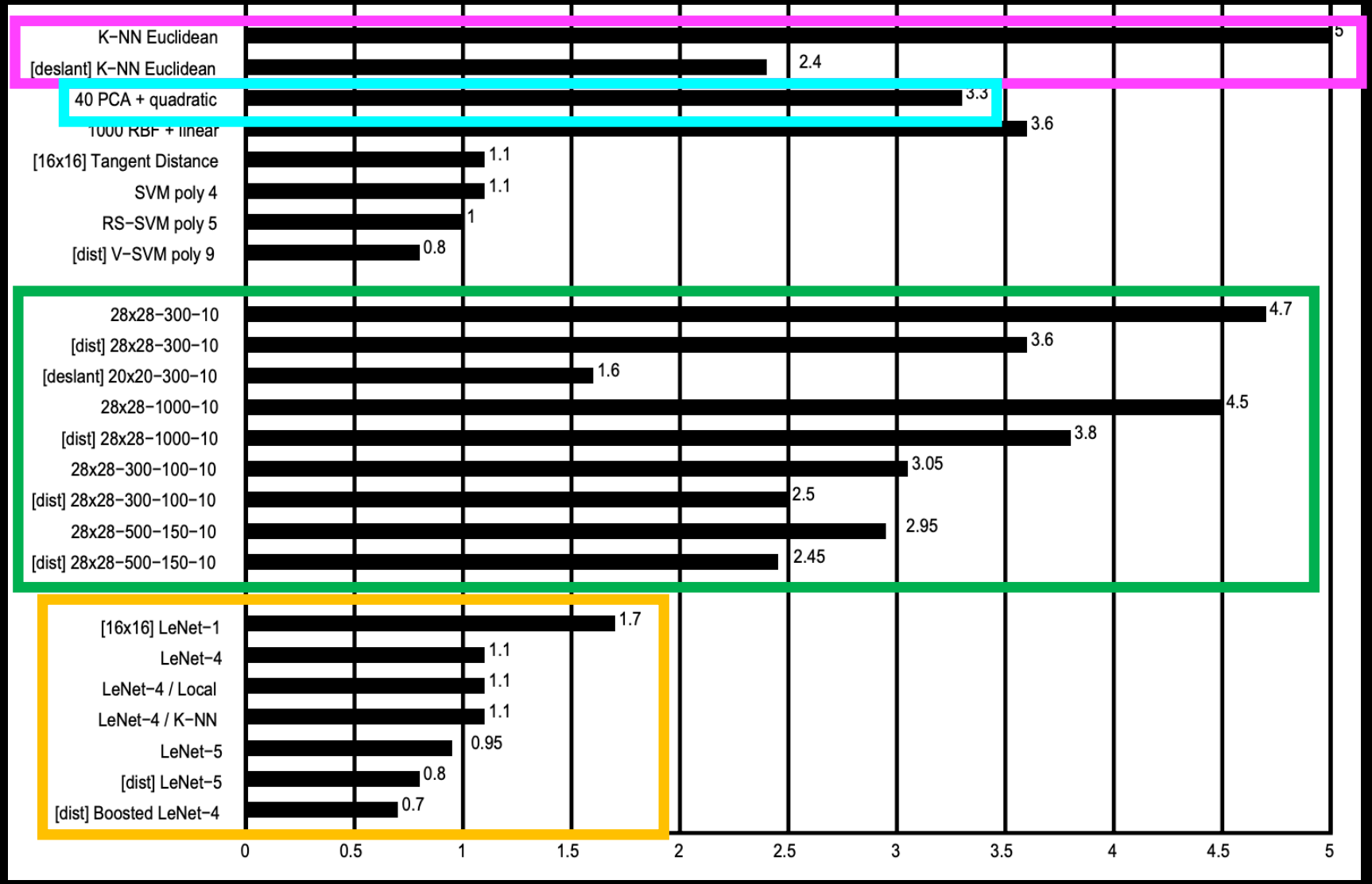
Why might mistakes occur?

- Ambiguity
- Insufficient representation in the training data



LeCun et al. Gradient-based learning applied to document recognition. 1998

LeNet-5 Analysis



The LeNet family was competitive with other available options, including those that are memory based (e.g., K-NN), use hand-crafted features, and use fully connected neural networks!

Higher capacity implementations of LeNet perform better!

LeCun et al. Gradient-based learning applied to document recognition. 1998

Today's Topics

- Issue: input shifts cause fully connected networks to fail
- Idea: convolutional neural networks (CNNs) designed for translation invariance
- Implementation: training CNNs end-to-end with gradient descent
- Real-world success: LeNet-5 for check reading
- Programming tutorial: convolutional neural networks in PyTorch

Today's Topics

- Issue: input shifts cause fully connected networks to fail
- Idea: convolutional neural networks (CNNs) designed for translation invariance
- Implementation: training CNNs end-to-end with gradient descent
- Real-world success: LeNet-5 for check reading
- Programming tutorial: convolutional neural networks in PyTorch

The End